altairsim — User Manual

2026-07-17 · f2355b9

What altairsim is

`altairsim` simulates the **MITS Altair 8800** and the **S-100 bus** it was built around.

It boots real software. Not software written to work with it — the actual artifacts, byte for byte, as they were sold: Altair BASIC off a cassette, CP/M 2.2 off an 8″ floppy, MITS Programming System II off paper tape's successor. None of it has been patched, and none of it knows it is not running on a real machine.

The point of a simulator is usually to run the old software. This one is built the other way round. It is a **bench for new hardware, and for the software that has to drive it** — which are not two jobs, they are one job, and it is very good at running the old software besides.

The S-100 bus is a real object in the program, not a wiring diagram implied by the code. Boards plug into it. They contend for addresses, pull the interrupt line, float the data bus when nobody is driving it, and get the answer wrong in exactly the ways real boards did. So a card you have not built yet can be *fitted* here, and the driver you have not finished can be *run* against it, months before either exists in copper.

That is the whole of the argument, and it is an argument about **where you find your bugs.** On real hardware a bug is a scope probe, a stubborn intermittent, and a machine that will not tell you what it just did. Here it is a breakpoint on a bus cycle. You can stop the machine mid-instruction, ask which card answered and which stayed silent, watch a driver poll a status bit that will never come true, and run the same thing again and get the same answer — because nothing here is intermittent, and nothing is hidden.

Every bug you kill on the bench is a bug you are not chasing at 2 MHz with a logic analyser. And when you do finally power up the real board, the software has already run — so the faults you are left with are the ones that are genuinely the *hardware's*: a timing margin, a noisy line, a pin on the wrong side of a buffer. That is a bring-up you can finish. The one where you cannot tell whether the board is wrong or the driver is wrong is the one that eats a month.

What it does

- **An 8080 that is validated, not merely plausible — and a Z80 alongside it.** TST8080, 8080PRE, CPUTEST and the full 8080EXM exerciser all pass — every one of the exerciser's CRC groups. Flags, carries, the undocumented behaviours, the lot. The Z80 clears the same bar, against ZEXDOC and ZEXALL.
- **Eleven board types**, all but one modelled from its own manual: two CPU cards (an 8080 and a Z80), RAM/ROM, two serial cards, a cassette interface, two floppy controllers, a

vectored-interrupt/real-time-clock card, the front panel, and one card of our own for moving files in and out.
- **A monitor** — the prompt you get when the machine is not running — with breakpoints (plain or conditional), single-stepping, disassembly, memory examine and deposit, a bus-cycle trace and a history ring, and a view of the bus itself: who decodes what, who is pulling which interrupt line, and where two cards are fighting.
- **Real I/O.** A serial card can be wired to your terminal, to a TCP socket (so you can telnet into the guest), or to an actual serial port on your machine, with the modem control lines wired through.
- **File transfer** between the host and CP/M, sandboxed. A card in the machine does the moving; the things you actually *type* — `HDIR`, `R` and `W` — are ordinary CP/M programs that live on a disk and run at the `A>` prompt, like `PIP` or `STAT`.

What it does not do

This section is here because a manual that only lists strengths is an advertisement.

- **It runs an 8080 or a Z80 — not an 8085.** Software that needs an 8085 will not run.
- **Six monitor commands are reserved but not built** — `SNAPSHOT`, `RESTORE`, `RECORD`, `REPLAY`, `EDIT` and `STOP`. They **resolve**: type `SN` and you are told that `SNAPSHOT` is waiting on the debugger, rather than being told nothing at all. That is deliberate — their abbreviations are claimed *now*, so that the day they land, `SN` does not silently stop meaning what your fingers think it means.
- **There is no snapshot and no replay.** You can trace the machine (`TRACE`) and read the run-up to a stop from the `HISTORY` ring, but you cannot save the machine's state and step backwards into it.
- **There is no video and no audio.** The Altair had neither. A terminal on a serial port is the display, exactly as it was.
- **Not every S-100 card is here.** The ones that are, are in the boards chapter. A Tarbell disk controller and a PMMI modem are designed but not built.
- **Timing is honest, but it is not a circuit simulation.** Instructions cost the right number of T-states and a cassette takes the right number of them to load. Propagation delays and analogue behaviour are not modelled, and no software from the period could tell.

What is in the box

You have the `altairsim` program, this manual, and some disks and tapes. The next chapter says what they are.

There is no installer, no configuration, and nothing to set up. Unzip it and run it.

What is in the package

```
altairsim               the program. One file, no dependencies, nothing to install.
altairsim-manual.pdf    this.
disks/                  disk images, each with the machine file that boots it.
tapes/                  cassette images, each with the machine file that boots it.
```

That is the whole distribution. There is no library to install, no runtime, and no configuration file you must write before the program will start.

The disks and tapes are *examples*, and each one is self-contained

Every example is a **directory**, and it holds both the media and the machine file that knows what to do with it:

```
disks/cpm22/cpm22-buffered.toml         the machine: a 56K Altair with a floppy controller
disks/cpm22/cpm22b23-56k.dsk            the disk that goes in it
```

Naming the machine file boots it:

```
$ altairsim disks/cpm22/cpm22-buffered.toml
```

The directory is the unit, and you may move it anywhere. A path written *inside* a machine file is resolved against **that file**, not against wherever you happened to be standing when you ran the program. So the machine file above names its disk as simply `cpm22b23-56k.dsk` — the one lying next to it — and the folder still boots after you copy it to your desktop, rename it, or mail it to somebody.

(The other half of that rule matters just as much: a path *you type* at the prompt is relative to **your shell**, because you are the one who can see your own directory. The two halves are covered in the machines chapter.)

The examples

Directory	What it is
`disks/cpm22`	**CP/M 2.2** on an 8″ floppy. This is the quick start.
`tapes/basic`	**Altair 4K BASIC 3.1** on a cassette, with the period bootstrap you toggle in to load it.

> **Altair Disk BASIC — not yet included.** The disk-BASIC example is still to be assembled. Where this manual describes it, it says so.

What is *not* in the package

The **source code** is not here. `altairsim` is an open project and the source is a separate thing to fetch; nothing in this manual requires it, and nothing in this manual refers to it.

The one exception worth naming: **if you want to build a board of your own** — which is what the simulator is really for — you need the source, and you want the *Developer Guide*, which is a different document. This one is about driving the machine, not extending it.

Running it

`altairsim` is a single executable. Unzip the package and run it from a terminal.

```
$ ./altairsim
altairsim 0.1.0 -- 8080, full speed.
machine: default.  HELP for commands.
altairsim>
```

That prompt is **the monitor**. The machine exists — it has memory, a processor, a console card and a floppy controller in it — but it is not running. Nothing has been started. This is the equivalent of standing in front of the real Altair with the power on and your hands on the switches.

You are not obliged to keep it in the current directory. Put it on your `PATH` and `altairsim` works from anywhere.

macOS: the first run

macOS marks anything that arrives from the internet and refuses to run it until you say otherwise. If you see "*cannot be opened because the developer cannot be verified*", clear the mark:

```
$ xattr -dr com.apple.quarantine ./altairsim
```

Do that once. It is not a comment on the program; it is what macOS does to every unsigned binary that arrives in a zip, whoever wrote it.

The mark is put there by whatever *fetched* the file — a browser, a mail client — so if you pulled the zip down with `curl` or `scp` there may be nothing to clear. The command says nothing and succeeds either way, which is why it is `-dr` and not `-d`: plain `-d` reports an error when the flag is already absent, and that error is not a problem.

Getting help, and getting out

Type	To
`HELP`	list every command
`HELP DUMP`	the usage and worked examples for one command
`QUIT`	leave

There is no `EXIT`. `QUIT` is the word, and `Q` is enough of it.

Commands resolve by **prefix**, so you type as much as it takes to be unambiguous and no more — `HELP` shows each command with its shortest form in brackets: `D[UMP]`, `DE[POSIT]`, `RES[ET]`. Type the part before the bracket. And **commands are not case-sensitive**, nor are the names of the cards in the machine; this manual writes them in capitals only because it is easier to read.

Which machine you get

Running `altairsim` with no arguments gives you a machine called `default` — a 56K Altair with a console and a floppy controller, which is the machine most period software expects.

Naming something gets you something else:

```
$ altairsim disks/cpm22/cpm22-buffered.toml     a machine file: this one boots CP/M
$ altairsim basic4k                     a BUILT-IN machine, by name
$ altairsim --list                      what the built-in names are
```

A **built-in** is a machine file that lives inside the program. There is nothing special about it — it is written in the same format as the ones in `disks/` and `tapes/`. To see what is actually in one, boot it and look:

```
$ altairsim -x BOARDS basic4k
```

…and if you want it as a file you can edit, `CONFIG SAVE mine.toml` writes out the machine you are actually running, and what it writes will boot.

The full story — every command-line option, and how `altairsim` decides whether a word is a built-in name or a filename — is in the machines chapter.

Quick start: CP/M in one command

```
$ altairsim disks/cpm22/cpm22-buffered.toml
```

That is the whole of it.

```
startup> RUN FF00
[console -- ^E returns to the monitor]

56K CP/M 2.2b v2.3
For Altair 8" Floppy

A>
```

You are in CP/M. It is 1977. Type `DIR`:

```
A>DIR
A: L80      COM : LADDER   COM : ED       COM : ASM      COM
A: DUMP     COM : XSUB     COM : PCGET    COM : LS       COM
A: SUBMIT   COM : LOAD     COM : SURVEY   COM : VIEW     COM
A: LADDER   DAT : LUNAR    BAS : M80      COM : MAC      COM
A: MBASIC   COM : PIP      COM : STAT     COM : DDT      COM
A: MOVCPM8  COM : NSWP     COM : SYSGEN   COM : ACOPY    COM
A: OTHELLO  COM : STARTRK  BAS : TICTAK   BAS : WM       COM
A: WM       HLP : CRC      COM : PCPUT    COM : AFORMAT  COM
A: STARINS  BAS : IOBYTE   TXT
A>
```

An assembler, a debugger, Microsoft BASIC, a text editor, and Star Trek. Run one:

```
A>MBASIC
```

What actually happened

Nothing was faked, and it is worth knowing what the one command did, because the rest of the manual is built on it.

The machine file named a **56K Altair with an 8″ floppy controller and a boot PROM at `FF00`**, put the disk image in drive 0, and then did one more thing: it typed `RUN FF00` for you. That is what the `startup>` line is telling you. **There is no `BOOT` command in this program** — booting a disk on an Altair meant setting the address switches to `FF00`, pressing EXAMINE, and then pressing RUN, so that is what the machine file says, in the operator's own words. (EXAMINE is

the step that matters: the switches by themselves change nothing, and it is EXAMINE that loads them into the program counter. `RUN FF00` is precisely those two presses — see the monitor chapter.) Anything you can type, a machine file can do; it gets no special powers.

From there it is all real: the PROM read sector 0 off track 0, that loader pulled CP/M into high memory and jumped into the BIOS, and the BIOS printed its banner.

Getting back out — `^E`

Press `^E` (Control-E). This is **ATTN**, and it is how you take the keyboard back from a running program:

```
A>
ATTN -- the machine is still at CA9C. RUN resumes.
C0Z1M0E1I0 A=00 B=007F D=CA01 H=BC0E S=BC37 IE=1 P=CA9C  CALL CA78
altairsim>
```

You are back at the monitor, and **the machine is stopped exactly where it stood**. The processor executes nothing while this prompt is up: the `P=CA9C` above is where it will still be in an hour. That is what makes the prompt useful — you can read memory, single-step, and set a breakpoint, and none of it is a moving target.

Stopped is not **lost**. ATTN is not RESET and it is not POWER: every register, every byte of memory, the disk in the drive and the guest's place in its own program are all exactly as they were. That is the whole content of "*the machine is still at CA9C*" — it is telling you the machine is intact and says where to pick it up.

Why it is not `^C`

Because `^C` **belongs to the guest.** CP/M reads `^C` — it is how you warm-boot it, and how you interrupt a BASIC program. A stop key that the guest also wants is a stop key the guest will eat, and then you are trapped inside your own simulator.

So the host intercepts `^E` **before the guest is ever offered the byte**. No program running inside the machine can disable it, ignore it, or take it away from you. Everything else on the keyboard — including `^C` — goes straight through to the guest, which is entitled to it.

(If `^E` collides with something you need, it moves: `CONSOLE attn=1D` makes it `^]`.)

Going back in — `RUN`

```
altairsim> RUN
```

That is all. The machine never stopped, so it simply picks up where it was, and your `A>` prompt is where you left it.

Leaving — `QUIT`

```
altairsim> QUIT
```

There is no `EXIT`. `Q` will do.

The three things to remember

`^E`	out of the guest, back to the monitor. The machine stops where it stands, and loses nothing.
`RUN`	back into the guest.
`QUIT`	done.

Careful: the disk is real, and there is no undo

The disk image is mounted **read/write**, because that is what a machine with a disk in it is. So anything you do in there *happens to the file on your host*, and nothing is keeping a copy.

You have two ways to be safe, and they answer different questions.

Write-protect the disk. `RO` refuses every write at the controller, so the file on your host cannot change no matter what the guest does:

```
altairsim> MOUNT dsk0:drive0 cpm22-buffered.dsk RO
```

In a machine file it is `readonly = true` in the drive's table. Use it when you want to *look around* — boot it, `DIR`, `TYPE`, run `STARTRK` — with the image guaranteed untouched. It is the stronger promise about your file, because it does not depend on you remembering anything.

But it is **read-only, and it means it**: the guest is not *told* the disk is protected, it is simply not allowed to write, and a program that sets out to write may not survive being refused. Mount `RO`

to read a disk, not to run a CP/M that expects to save your work. The disks chapter says why the guest cannot be told.

Or copy the folder, which is what you want the moment you intend to actually *write* — save a BASIC program, assemble something, use `PIP`. It is a directory with a machine file and a disk image in it, and it boots from anywhere:

```
$ cp -R disks/cpm22 my-cpm
$ altairsim my-cpm/cpm22-buffered.toml
```

The disks chapter has both in full.

There is one more trap, and it is not obvious: **this BIOS does not write to the disk when CP/M closes a file.** It writes into a track buffer in memory and flushes it the next time it reads the console. So do not kill the program the instant a file operation finishes — **get back to the `A>` prompt first**, and the directory update will have landed. The disks chapter explains why.

No disk? Start with a tape instead

If you would rather see something boot from nothing at all — no disk, no PROM, the bootstrap toggled in by hand exactly as MITS printed it in the manual — turn to the tapes chapter and load Altair 4K BASIC 3.1. It is the more instructive machine, and it is the one that shows you what an Altair actually was.

Quick reference

Getting out, and back in

Key	Does
`^E`	**ATTN** — stop the machine and take the keyboard back. Nothing is lost.
`RUN`	Resume, at the exact instruction it stopped on.
`QUIT`	Leave. (There is no `EXIT`.)

Command line

```
altairsim [options] [machine]

  machine            a built-in name, or a config file (has a '/' or ends .toml).
                     Omitted: ./altairsim.toml if there is one, else `default`.
  -m, --machine <n>  ALWAYS a built-in name -- never a file.
  -f, --file <path>  ALWAYS a file -- never a built-in name.
  -n, --none         empty backplane: no boards, no memory, nothing.
  -l, --list         list the built-in machines and exit.
  -s, --script <f>   run a command script, then exit with its status.
  -x, --exec <cmd>   run one monitor command (repeatable), then exit.
  -i, --interactive  after --script/--exec, stay in the monitor.
      --mcp          MCP server on stdio.
  -v, --version      -h, --help
```

Monitor commands

Type the part before the bracket. `*` = resolves, but not built yet.

Command	Usage
`D[UMP]`	`DUMP [<addr>|<range>] [WIDTH=16]`
`S[TEP]`	`STEP [n]`
`N[EXT]`	`NEXT`
`R[UN]`	`RUN [addr]`
`H[ISTORY]`	`HISTORY [n]`
`M[OUNT]`	`MOUNT <id>[:<u>] <file> [WP]`
`B[REAK]`	`BREAK [<addr> [IF <expr>] | MEM R|W <addr> | IO R|W <port>] [TRACE ON|OFF]`
`E[DIT]` *	`EDIT <addr>  -- interactive; Enter advances`
`C[ONFIG]`	`CONFIG LOAD <f.toml> | CONFIG SAVE <f.toml>`
`SE[T]`	`SET <id> <k>=<v>`
`SH[OW]`	`SHOW <id>|BUS [MAP|IO|IRQ|CONTENTION]|ROMS|MOUNTS|PATHS|CONSOLE|SYMBOLS|MACHINE`
`DE[POSIT]`	`DEPOSIT <addr> <bytes...>`
`EX[AMINE]`	`EXAMINE [<addr>]`
`I[N]`	`IN <port>`
`O[UT]`	`OUT <port> <byte>`
`L[OAD]`	`LOAD <file> [AT <addr>] [FORMAT=BIN|HEX] [ROM]`
`SA[VE]`	`SAVE <file> <range> [FORMAT=BIN|HEX]`
`F[ILL]`	`FILL <range> <byte>`
`SEA[RCH]`	`SEARCH <range> <bytes...>|"str"`
`COM[PARE]`	`COMPARE <range> <addr>|<file>`
`MOV[E]`	`MOVE <range> <dest>`
`W[HO]`	`WHO <addr> | WHO IO <port>`
`BO[ARDS]`	`BOARDS [LIST]|TYPES|ADD <type> <id> [k=v...]|REMOVE <id>`
`RE[GS]`	`REGS | SET REG <r>=<v>`
`REGI[ON]`	`REGION ADD <id> type=ram|rom at=<addr> [size=|mount=]`
`DI[SASM]`	`DISASM [<addr>|<range>] [n] [CPU=8080]`
`SY[MBOLS]`	`SYMBOLS LOAD <file> [REPLACE] | SYMBOLS CLEAR`
`U[NMOUNT]`	`UNMOUNT <id>:<u>`
`DISC[ONNECT]`	`DISCONNECT <id>:<u>`
`CONS[OLE]`	`CONSOLE [<k>=<v>...]`

Command	Usage
`CONN[ECT]`	`CONNECT <id>:<u> <endpoint>`
`RES[ET]`	`RESET [CPU]`
`P[OWER]`	`POWER`
`T[RACE]`	`TRACE ON|OFF [file] [MASK=IN,OUT,IRQ,DMA,CONTENTION]`
`STO[P]` *	`STOP`
`SN[APSHOT]` *	`SNAPSHOT <file>`
`REST[ORE]` *	`RESTORE <file>`
`REC[ORD]` *	`RECORD <file>`
`REP[LAY]` *	`REPLAY <file>`
`NO[BREAK]`	`NOBREAK [id]`
`HE[LP]`	`HELP [<command>]`
`Q[UIT]`	`QUIT`

Boards

Type	What it is
`memory`	RAM/ROM card: a list of regions, PHANTOM*, and five banking schemes
`8080`	MITS 88-CPU: an 8080A at 2 MHz. Decodes nothing -- it drives the bus
`z80`	Generic Z80 CPU card. Decodes nothing -- it drives the bus. The 88-CPU's twin, with a Z80 core
`2sio`	MITS 88-2SIO: two 6850 ACIAs, units 'a' and 'b'. Four ports at BASE+0..3
`sio`	MITS 88-SIO: one COM2502 UART, unit 'tty'. Two ports at BASE+0..1. INVERTED status bits
`dcdd`	MITS 88-DCDD: 8" hard-sector floppy, up to 16 drives. Three ports at BASE+0..2. INVERTED status bits
`mds`	MITS 88-MDS: 5.25" minidisk, 4 drives. Same three ports as the dcdd -- but 300 RPM, 64 us/byte, and a motor that stops after 6.4 s
`acr`	MITS 88-ACR: cassette. An 88-SIO B + an FSK modem, unit 'tape'. Brings the REWIND verb
`fp`	Altair front panel: the SENSE switches at port FF (read-only), and the lamps
`virtc`	MITS 88-VI/RTC: vectored interrupts (VI0-VI7 -> RST n) and a real-time clock. One port at FE
`hostbridge`	Host Bridge: guest <-> host file transfer, sandboxed. OUR OWN CARD, not a period one. Two ports at BASE+0..1. R.COM/W.COM/HDIR.COM

Machines

Machine	What it is
`4k`	The Altair as it actually left Albuquerque.
`altmon`	An Altair with a monitor in ROM and a terminal on it.
`basic4k`	The machine Altair 4K BASIC was sold to run on: a cassette in the ACR, a Teletype
`basic8k`	The machine Altair 8K BASIC was sold to run on: a cassette in the ACR, and a terminal
`default`	The machine you get when you name none: 56K, and the DBL boot PROM at FF00.
`minidisk`	The Altair Minidisk: an 88-MDS at 08, the MDBL boot PROM, and CP/M 2.2b on a 5.25" disk.
`ps2`	The machine MITS Programming System II ran on. It is `basic8k`'s CARDS -- same 2SIO, same
`ps2int`	MITS Programming System II, WITH INTERRUPTS. `ps2` with A9 down and an 88-VI/RTC in it.
`z80`	A minimal Z80 machine: a `z80` CPU, 64K of RAM, and a 2SIO console.

A machine file, in one look

```
[machine]
name    = "mine"
base    = "default"        # start from a machine, and say what is DIFFERENT
startup = ["RUN FF00"]     # the operator's own keystrokes. There is no BOOT verb.

[[board]]                  # type + a NEW id      -> ADD the card
type = "2sio"              # type + an id from the base -> REPLACE it outright
id   = "sio0"              # NO type + an id      -> MODIFY the one already there
port = 10                  # remove = true        -> PULL THE CARD OUT

  [board.unit.a]           # a unit's own settings
  connect = "console"

  [[board.region]]         # a list the card owns (memory)
  type = "ram"
  at   = 0000              # hex: it is an address
  size = "56K"             # decimal: it is a size

  [[board.drive]]          # a list the card owns (disk controllers)
  unit  = 0
  mount = "cpm.dsk"        # relative to THIS FILE

[console]                  # the HOST's terminal -- not a board
strip7out = true
```

Paths: a path *inside* a machine file is relative to **that file**. A path you *type* is relative to **your shell**.

Endpoints — `CONNECT <id>:<unit> <endpoint>`

Endpoint	Is
`console`	the host terminal. Exactly one unit may hold it.
`null`	nowhere. Writes vanish, reads never come.
`loopback`	itself — what you write comes back.
`socket:PORT`	**listens** — this is telnet-in.
`socket:HOST:PORT`	**calls out**.
`serial:DEVICE`	a real serial port on this host.

The monitor

The `altairsim>` prompt is **the monitor**: the front panel of the machine, and its debugger, which here are the same thing. Everything the front panel of a real Altair could do, the monitor can do — and a great deal it could not. It breakpoints, it single-steps, it disassembles, and it will show you the bus itself: who decodes what, who is pulling which interrupt line, and where two cards are fighting over an address. That is the **debugging** chapter, and it is most of why this program exists.

What it is not is a *menu* — a layer sitting between you and the machine, offering a fixed set of things it is prepared to let you inspect. There is no debug mode to enter and nothing is watching from the outside. A breakpoint is **the machine stopping**, not a script noticing that it should have. `IN` and `OUT` run **real bus cycles**, with every side effect a real one has — read a UART's data port at this prompt and you have taken the byte, exactly as the guest would have. The panel and the debugger are one object because on an Altair they were one object: a man at the switches, reading lamps.

The machine does not have to be running for the monitor to work. Most of what follows — examining memory, running a bus cycle, fitting a card — works on a machine with the power on and the processor idle, which is exactly the arrangement the front panel was for.

Commands resolve by prefix

There are **no aliases and no memorised abbreviations**. You type as much of a command as it takes to be unambiguous, and the first command that matches wins.

`HELP` prints the whole menu with each command's shortest form in brackets:

```
BO[ARDS]          B[REAK]           COM[PARE]         C[ONFIG]
CONN[ECT]         CONS[OLE]         DE[POSIT]         DI[SASM]
DISC[ONNECT]      D[UMP]            E[DIT]*           EX[AMINE]
F[ILL]            HE[LP]            H[ISTORY]         I[N]
L[OAD]            M[OUNT]           MOV[E]            N[EXT]
NO[BREAK]         O[UT]             P[OWER]           Q[UIT]
REC[ORD]*         REGI[ON]          RE[GS]            REP[LAY]*
RES[ET]           REST[ORE]*        R[UN]             SA[VE]
SEA[RCH]          SE[T]             SH[OW]            SN[APSHOT]*
S[TEP]            STO[P]*           T[RACE]           U[NMOUNT]
W[HO]
```

Type the part before the bracket. `D` is `DUMP`; `DE` is `DEPOSIT`; `RES` is `RESET`. Case does not matter, here or in the name of any card.

The `*` marks the six commands that **resolve but are not built yet**. Type `SN` and it will tell you that `SNAPSHOT` is waiting on the debugger. That is on purpose: their abbreviations are claimed *now*, so the day they land, `SN` does not quietly stop meaning what your fingers have learned it means. An abbreviation is a contract.

`HELP <command>` gives you the usage and worked examples for one of them, and `?` is the same as `HELP`.

> **`R` is `RUN`, not `RESET`.** It is the command you type every session, and it is the one that costs nothing if you did not mean it. A bare `R` that reset the machine would be one you had to set up again. `RESET` pays the letters: `RES`.

Numbers: one rule, and it is not negotiable

> **On the wire → hex. Never on the wire → decimal.**

If the 8080 can see it, it is **hex**: an address, a port, a data byte, a register. If it never leaves your head, it is **decimal**: a count, a width, a size, a drive number.

```
DUMP 100            address  -> 0100 hex
STEP 10             a count  -> ten instructions
OUT FF 55           port and byte -> both hex
SET sio0:a baud=9600      a baud rate -> nine thousand six hundred
```

You can always force the issue: `0x`, `$` and a trailing `h` force hex; a leading `#` forces decimal; and a `K` or `M` suffix is **always** decimal (`48K` is 49,152 — so `0x10K` is a contradiction and is rejected rather than guessed at).

This rule is the same everywhere — in the monitor, in a machine file, and in every board's settings. There is no second convention to learn.

Naming a card: `<id>[:<unit>]`

Every card in the machine has an **id** you chose (`cpu0`, `sio0`, `dsk0`), and some cards have **units** inside them — the two channels of a serial card, the four drives on a floppy controller, the ROM socket on a memory card.

```
SHOW sio0              the card
SET  sio0:a baud=1200  one channel of it
MOUNT dsk0:drive1 my.dsk
```

You may leave out anything that carries no information. If there is only one floppy controller in the machine, `dsk` will find it. If a card has only one thing you could mount into, you need not name it — `MOUNT ACR tape.bin` puts a cassette in the one recorder.

But anything **genuinely plural you must say**. There are four drives on that controller and the machine will not guess which one you meant; it will tell you so and stop.

Seeing the machine

```
altairsim> BOARDS
  ID    TYPE    I/O       UNITS                       MEMORY
  ----  ------  --------  --------------------------  ------------------------------
  fp0   fp      FF        -                           -
  cpu0  8080    -         1 cpu: 8080                 -
  sio0  2sio    10,12     2 serial: a*, b             -
  dsk0  dcdd    08,09,0A  4 disk: drive0(empty), ...  -
  mem0  memory  -         1 rom: rom0                 0000-DFFF  ram  56K
                                                      FF00-FFFF  rom  dbl  phantom:all

  * holds the console
```

That is the backplane: what is plugged in, what ports each card answers to, what is in its units, and what it decodes in memory.

Command	Shows
`BOARDS`	the backplane
`SHOW <id>`	one card: every setting, its value, and what it will accept
`SHOW MACHINE`	the whole machine
`SHOW CONSOLE`	which unit holds your keyboard, and how bytes are being transformed
`SHOW BUS MAP`	who decodes which addresses — and what floats
`SHOW BUS IO`	who decodes which ports
`SHOW BUS IRQ`	who is strapped to which interrupt line, and who is pulling it
`SHOW BUS CONTENTION`	where two cards are fighting

`SHOW <id>` is worth dwelling on, because it is the **only** thing you need in order to configure a card. It lists every property, what it is set to, and what values are legal — and those property

names **are** the keys you write in a machine file. There is no second schema anywhere in this program. The board reference at the back of this manual is printed from the same source.

Changing the machine

```
SET cpu0 clock_hz=2000000      give it the real 2 MHz crystal
SET mem0 fill=zero             RAM comes up zeroed instead of random
SET fp0  sense=80              set the SENSE switches
BOARDS ADD 2sio sio1 port=20   fit a second serial card
BOARDS REMOVE sio1             pull it out
CONFIG SAVE mine.toml          write out the machine you are actually running
```

`CONFIG SAVE` round-trips: what it writes, `altairsim mine.toml` will boot.

Running, and stopping

```
RUN FF00     load the PC and go — the same two motions as the panel's switches
RUN          carry on from wherever the processor is
```

`RUN <addr>` is EXAMINE followed by RUN, exactly as you would do it on the front panel. There is no `BOOT` command in this program, and there should not be: a machine that ought to start says so with the operator's own keystroke.

If a card holds the console, **the guest gets the keyboard** — every key, including `^C`, which a CP/M program is entitled to read.

ATTN takes it back

`^E` is ATTN. The host intercepts it *before the guest is ever offered the byte*, so no program running inside the machine can disable it, trap it, or take it from you.

```
A>
ATTN -- the machine is still at CA9C. RUN resumes.
altairsim>
```

ATTN stops the machine and gives you the monitor. Nothing executes while this prompt is up. But it stops the machine without *disturbing* it — ATTN is not RESET and not POWER, so the registers, the memory and the disk are exactly as the guest left them, and a bare `RUN` (no address) picks up at the very instruction it was about to execute. That is what "*still at CA9C*" is telling you.

`CONSOLE attn=1D` moves ATTN to `^]` if `^E` collides with something the guest wants.

What stops it for real

A `RUN` ends when it hits a **breakpoint**, or a `HLT` that nothing can wake — and it always says which. With no console connected there is nothing to hand the keyboard to, so it simply runs, and `^C` stops it.

Speed

It runs flat out by default. `clock_hz` on the CPU card is `0`, which means "as fast as this host can go", and a cassette that took a real Altair 110 seconds comes off in about one.

```
SET cpu0 clock_hz=2000000
```

…buys back the 2 MHz machine, and with it the 110 seconds. **What the guest sees is identical either way** — the tape still costs the same number of T-states, the timing loops still count the same. The crystal buys period *feel*, not period *behaviour*.

`SHOW cpu0` reports back what it actually reached. Beside `clock_hz` — the crystal you asked for — sits `achieved_hz`, the clock the run loop hit the last time it ran. Flat out, that is how fast this host went; with a crystal set, it is how close the pacing landed. It reads `0` until the machine has run, and you cannot set it — it is a measurement, not a knob.

Until the guest talks to something outside the machine. A program measures time by counting instructions, so at `clock_hz = 0` a timeout it believes is three seconds can expire in thirty milliseconds of yours — which is why an XMODEM transfer to your host wants the real crystal, and a cassette does not. See the troubleshooting chapter.

RESET is not POWER

`RESET`	the bus's RESET* line. The processor restarts at `0000`. **Memory survives**, disks stay mounted.
`POWER`	a power cycle. **This is the only thing that loses RAM** and re-reads the ROM images.

`RESET` does not clear memory because pressing RESET on a real Altair did not clear memory. That is not a simplification; it is the behaviour a lot of period software depends on.

RESET* is a wire, and every card is listening

The processor is not the only thing that hears it. `RESET*` is a **line on the backplane**, and it runs past every card in the machine — so `RESET` is not an instruction the simulator carries out on your behalf. It is a signal put on the bus, and each board answers it the way its own silicon answered it, which is not the same answer twice:

- The **memory card** clears its bank latch — and does not touch one byte of RAM. A RAM chip has no reset pin to touch it with.
- The **floppy controller** flushes the sector it was in the middle of writing, deselects the drive, and lets the head unload. On the 5¼″ minidisk the motor spins down; on the 8″ drive nothing spins down, because that card has no motor control to spin down *with*.
- The **2SIO does nothing at all**, and that is the most instructive answer of the three. The MC6850 has **no reset pin** — Vss, RxD, RxCLK, TxCLK, RTS, TxD, IRQ, CS0–CS2, RS, Vcc, R/W, E, D0–D7, /DCD, /CTS, and that is the entire package. `RESET*` reaches the card and has **nowhere to land**, so the baud rate, the word format, RTS and the interrupt enables all survive a reset, exactly as they do on the bench. (The 6850's *master reset* is a real thing, but it belongs to the **guest**: a program performs it by writing to the control register. The front panel cannot do it for you.)

Which is why a reset here behaves like a reset there. Hit `RESET` in the middle of a disk write and you get what the hardware gave you: a half-written sector on the disk, a serial port still configured exactly as the dead program left it, and every byte of your RAM intact and waiting to be looked at. Nothing is tidied up on the way out, because on a real machine nobody was there to tidy it.

And `POWER` is a *different wire*

This is the part that makes `POWER` a different event rather than a bigger one. Switching the machine on drives **`POC*`** — Power-On Clear, its own line on the backplane — and a card is free to treat the two lines differently, because the real cards did.

The **88-VI/RTC** is the case that proves it. Its manual is explicit that POC disables every function on the board, and the schematic runs POC — and *only* POC — to that logic. `RESET*` is not wired to it at all. So an interrupt controller that a crashed program left armed and enabled **stays armed through a `RESET`**, and comes back only when you `POWER` the machine. That is not our shortcut; it is the card, and it is why a program that resets its way out of trouble can still be taking interrupts it forgot it asked for.

`POC*` is also the only moment RAM is allowed to forget. On `POWER` the memory card refills itself — with **random bytes by default**, because static RAM does not come up zeroed, and a simulator that quietly zeroes it will never once catch the program that assumed otherwise — and re-reads every ROM image from disk.

	The processor	The boards	RAM
`RESET`	restarts at `0000`	`RESET*` on the bus; each card answers as its silicon did — some do nothing	**survives**
`POWER`	restarts at `0000`	`POC*` on the bus; the cards come up as they do from cold	**refilled**, ROMs re-read

Debugging

This is the chapter the simulator exists for.

Running old software is the easy half. The hard half is being able to see what a machine is actually *doing* — which card answered, what went out on the bus, why the interrupt never arrived — and that is what the commands in this chapter are for. Roughly fifteen of the monitor's forty commands are here.

Where the processor is — `REGS`

`REGS` prints the whole processor on one line. The flags come first — carry, zero, minus, even parity, interdigit carry — then the register pairs, the stack pointer, the interrupt-enable flip-flop, and the program counter. The last column is **the instruction the processor is about to execute**, already disassembled.

You get this line free every time the machine stops, so most of the time you never type `REGS` at all.

```
altairsim> REGS
C0Z1M0E1I0 A=00 B=007F D=CA01 H=BC0E S=BC37 IE=1 P=CA9C  CALL CA78
```

Flags are registers, and you may set them:

```
SET REG A=3F
SET REG CY=1
```

Stepping — `STEP`

`STEP` runs **real bus cycles through the real instruction decode**. It is not an interpreter running alongside the machine — it *is* the machine, moved forward by one instruction. It prints each instruction as it goes; past thirty-two it runs quietly and tells you where it ended up.

```
STEP        one instruction
STEP 20     twenty of them (a count, so it is decimal)
```

`NEXT` steps *over* a subroutine. At a `CALL` or `RST`, `STEP` walks you down into the callee — every instruction it runs, and everything it in turn calls. Often you do not care: the routine works, and you want the *next* instruction in the code you are reading, not a tour of a print routine. `NEXT` gives you that. On a `CALL` or `RST` it runs the callee at full speed and stops the

instant it returns; on anything else it is just a single step. It does exactly what you would do by hand — sets a breakpoint at the return address and runs to it — so the callee is live while it runs: it can read the console, and `^E` (ATTN) or `^C` stops it if it never comes back. A breakpoint that fires *inside* the callee stops you there, as it should.

```
NEXT        over the CALL/RST at PC, else one instruction
N           the same -- it owns the letter, because you type it constantly
```

Breakpoints — `BREAK`, `NOBREAK`

There are two kinds, and only the first is about the processor at all.

`BREAK MEM` and `BREAK IO` watch bus cycles, not instructions. That is a much stronger thing, and it is the reason to prefer them. A memory watch will catch a DMA transfer that no instruction on the CPU ever performed, and it will work unchanged on any processor you put in the machine, because it is watching the backplane rather than the program.

If you are chasing a byte that keeps getting clobbered, `BREAK MEM W <addr>` will find who is doing it, whatever is doing it.

```
BREAK FF13            stop when the PC gets there
BREAK 2C00-2CFF       ...or anywhere in a range
BREAK MEM W 100       stop when ANYTHING writes to 0100
BREAK IO  R 10        stop on an IN from port 10
BREAK                 list them
NOBREAK 2             clear one (the id is a count, so decimal)
NOBREAK               clear them all
```

An address breakpoint can carry a condition. `BREAK <addr> IF <expr>` stops only when the expression is true — the registers, tested the moment the PC reaches the address. It is what you reach for when a breakpoint fires ten thousand times before the once you care about: put the distinguishing state in the condition and let the machine run until it holds.

A bare word that names a register *is* that register, so a literal needs a leading zero — `0A` is ten, `A` is the accumulator. `==` `!=` `<` `>` `<=` `>=` compare, `&&` `||` combine, `&` `|` mask, and parentheses group. Only a plain address breakpoint takes a condition: the `MEM`/`IO` watches fire in the middle of a cycle, where a register read has no boundary-consistent answer.

```
BREAK 100 IF A==0
BREAK 100 IF HL==8000 && Z==1
BREAK 100 IF (A&0F)==0        only when the low nibble is zero
```

Reading a block of memory — `DUMP`

`DUMP` is how you read a lot of memory at once. A bare `DUMP <addr>` runs to the **end of its page**, and a bare `DUMP` carries on from there — so however you first landed, the rows stay page-aligned and the columns never move under your eye.

It only *looks*. Nothing is consumed and no bus cycle is run.

```
DUMP 100          0100-01FF: a whole page
DUMP              the next page
DUMP FF00-FF0F    exactly that range
DUMP 100/20       0100-011F  (a length, and it is part of the address, so it is hex)
DUMP 0 WIDTH=8    eight bytes to a line (a count: decimal)
```

One byte at a time — `EXAMINE`, `DEPOSIT`

These two are **the front panel's switches**, and they behave like them. `EXAMINE` shows a single byte — hex, ASCII, and its bits — and a bare `EXAMINE` steps to the next one, which is the panel's EXAMINE NEXT.

`DEPOSIT` runs a **real bus write**. If no card decodes that address, it says so rather than pretending to have stored something — which is the difference between a debugger and a notepad.

```
EXAMINE 2C00      one byte: hex, ASCII, and its bits
EXAMINE           the next one — the panel's EXAMINE NEXT
DEPOSIT 100 C3 00 2C
```

Disassembling — `DISASM`

`DISASM` **peeks**: it reads memory without running a bus cycle. That matters, and it is not a detail. A `read()` on a serial card *consumes* a byte from its receiver, and a disassembler that ate the guest's input while you were looking at it would be a debugger you could not trust. Nothing in this chapter that only *looks* at memory will disturb it.

```
DISASM FF00       sixteen instructions
DISASM            carry on
DISASM 0-2F       exactly that range
```

Symbols — `SYMBOLS`, `SHOW SYMBOLS`

Everything so far has spoken in hex. Load an assembler's symbols and you can name things instead: `BREAK START` rather than `BREAK 0100`, `DUMP MSG/20`, `EXAMINE BDOS`. A symbol is accepted anywhere an address is typed, and in a `BREAK … IF` condition.

```
SYMBOLS LOAD prog.SYM              a symbol table
SYMBOLS LOAD roms/ALTMON/ALTMON.PRN   ...or an assembler listing
BREAK START
DUMP MSG/20
BREAK 200 IF HL==STACK
SHOW SYMBOLS                       all of them
SHOW SYMBOLS SIO*                  filtered by a glob
SYMBOLS CLEAR                      forget them
```

Two kinds of file, and the toolchains that write them. A `.SYM` is a flat list of name = value. Two toolchains write one: Digital Research's `MAC` / `RMAC` assemblers (every symbol, read by `SID`), and — with the right switches — Microsoft's **L80** linker. `L80`'s `/M` prints a *map* to the console, but `filename/N/Y/E` writes a real `filename.SYM`; the catch is that an `L80` `.SYM` holds **globals only** (the `PUBLIC` names), so a module's local labels and `EQU`s are not in it. For those, use the assembler's listing. A `.PRN` or `.LST` is the assembler's own listing — from CP/M `ASM`, Microsoft `M80`, or `MAC` — and it is the richer source, because it marks an `EQU` and so can tell a constant apart from a program label: only real labels are offered back as addresses, so `0005` never starts printing as `BDOS`.

Addresses must be absolute. A relocatable `M80` listing marks its addresses, and loading one is refused by the offending line — link it and load the `.SYM`, or assemble to an absolute origin. A `.SYM` is written after linking and is absolute already, so it never has this problem.

Symbols are yours, not the machine's. Like a breakpoint, the table is the debugger's view, not part of any card — it survives `RESET`, `POWER`, and `CONFIG LOAD`, and `SYMBOLS CLEAR` is its `NOBREAK`. Loading two files **merges** them (the newest of a clashing name wins, and the command says how many were redefined); `SYMBOLS LOAD <file> REPLACE` starts fresh. A machine file can name a symbol file in its `startup`, and `CONFIG SAVE` writes the filename back out — the file, not the parsed table, exactly as it does for a built-in ROM.

A name beats a hex literal. If a symbol is spelled like a number — `FACE`, `BEEF` — the symbol wins; write `0FACE` (or `$FACE`) to force the number, the same escape that tells the register `A` from the number `0A`.

Searching, filling, moving

The block operations. `COMPARE` will take a file as its second operand, which is how you check what the machine loaded against what you meant to load.

```
SEARCH 0-FFFF C3 00 2C      find those bytes
SEARCH 0-FFFF "BDOS"        ...or that string
FILL 100-1FF 00
MOVE 100-1FF 2000
COMPARE 100-1FF 2000        ...or against a file
```

Running real bus cycles by hand — `IN`, `OUT`

These are not simulated reads. **`IN` runs an input cycle on the bus, with every side effect a real one would have** — it will consume a character from a UART's receiver, it will advance a disk controller's sector counter. That is the point of them: it is how you poke a card the way the guest's software would, without writing any guest software.

```
IN  10           run a real IN cycle on port 10
OUT FF 55        run a real OUT cycle
```

Asking without touching — `WHO`

`WHO` asks who *would* answer. **No cycle is run and nothing is consumed.** It is the question you want when `IN 10` gives you `FF` and you cannot tell whether that is data or whether nothing is there at all.

It reports contention, and it reports `PHANTOM*` — so if two cards are fighting, or if one card has switched another one off, `WHO` is where you find out.

```
altairsim> WHO IO FF
port FF OUT: nobody (an OUT here goes nowhere)

WHO 2C00         who decodes this address?
WHO IO 08        who decodes this port?
```

FF is not data

An `IN` from a port nothing decodes returns `FF`. So does a read from an address no card answers for.

That is not an error code and it is not a convention we invented — it is what a **floating bus** reads. Nobody is driving the data lines, they idle high, and the processor faithfully reads eight ones. A real Altair does exactly this.

It has a famous consequence, and it is worth knowing because you will meet it: on a machine with no interrupt-vector card, a board pulls the interrupt line, nobody drives the data bus during the acknowledge cycle, the processor reads `FF` — and `FF` is `RST 7`. That is not a fallback anybody coded. It is what the hardware does, and it is why the interrupt vector on a bare Altair is `RST 7`.

So when you see `FF`, ask `WHO`.

Looking at the bus itself — `SHOW BUS`

Where `WHO` asks about one address, `SHOW BUS` shows you the whole backplane at once.

`SHOW BUS IRQ` is the only window onto the interrupt wiring, and interrupt wiring is the part of a machine you cannot see. A card strapped to a line that nothing listens to fails in total silence — the software just never gets its interrupt, and there is nothing to look at. This command is what makes that visible.

`SHOW BUS CONTENTION` is the one to reach for when a machine you built yourself is misbehaving for no reason. Two cards decoding the same port is a real hardware fault, and the simulator will not quietly pick a winner for you.

```
SHOW BUS MAP          who decodes what in memory — and what floats
SHOW BUS IO           who decodes which ports
SHOW BUS IRQ          the eight interrupt lines: who is strapped where, who is pulling
SHOW BUS CONTENTION   where two cards answer the same thing
```

The bus over time — `TRACE`, `HISTORY`

`WHO` and `SHOW BUS` are snapshots — the backplane as it is *now*. `TRACE` and `HISTORY` show you the same backplane over *time*, which is what you want when the bug is not where the machine stopped but somewhere in how it got there.

`HISTORY` is a flight recorder. A fixed-size ring of the most recent bus cycles is always filling while the machine runs, so when a breakpoint fires — or the machine wanders off into the weeds — the run-up to it is *already* recorded. You do not arm it; it is on. A bare `HISTORY` shows the last sixteen cycles, oldest first; `HISTORY <n>` shows the last *n*. Each line is a cycle, not an instruction, so a DMA transfer's cycles are in there too.

```
HISTORY              the last 16 cycles
HISTORY 100          the last hundred (a count, so decimal)
```

`TRACE` logs every cycle as it happens — to the console, or to a file. Like the breakpoints that watch the bus, it is not a CPU feature: it watches the same stream every card sees, so it works unchanged on any processor you put in the machine. A `MASK` keeps only the categories you name, and no mask keeps them all: `IN`, `OUT`, `IRQ`, `DMA`, `CONTENTION`. A cycle survives the mask if it matches any of them.

```
TRACE ON                    every cycle, to the console
TRACE ON run.log            ...to a file instead
TRACE ON MASK=IN,OUT        just the port traffic
TRACE OFF                   stop tracing
```

Tracepoints — tracing one part of a program

A whole-program trace is a firehose. A mask narrows it by *category*, but often you do not want a category — you want a *place*: this subroutine, and nothing else.

That is a tracepoint. Add `TRACE ON` or `TRACE OFF` to a `BREAK` and it stops being a breakpoint: instead of stopping the machine it flips the trace, and the machine runs on. Two of them bracket a region.

```
altairsim> BREAK 2C00 TRACE ON     start tracing when PC reaches 2C00
altairsim> BREAK 2C40 TRACE OFF    ...and stop again at 2C40
altairsim> RUN FF00
```

`TRACE ON` traces the instruction *at* its address; `TRACE OFF` does not. The region is `[on, off)` — exactly the half-open range you would write down if someone asked you which instructions were in the subroutine.

Because a trace toggle reads no registers, it works on the bus kinds too — where `IF` cannot go. And the cycle that triggered it is the *first line* of the trace, not the line above it: a trace should show its own reason.

```
altairsim> BREAK MEM W 2000 TRACE ON    trace onward from whatever writes 2000
```

They compose with `IF`, and they still do not stop:

```
altairsim> BREAK 200 IF HL==8000 TRACE ON
```

Where the trace goes is `TRACE`'s business, not the tracepoint's. A tracepoint that has never been told anything traces to the console. To send it to a file, configure it first — and this is what `TRACE OFF` is for: it stops the tracing but *remembers where it was going*, so a tracepoint can pick it up again.

```
altairsim> TRACE ON run.log MASK=DMA    configure it: file, mask — and it starts
altairsim> TRACE OFF                    stop, but the file and mask are remembered
altairsim> BREAK 2C00 TRACE ON          arm the region
altairsim> BREAK 2C40 TRACE OFF
altairsim> RUN FF00                     run.log gets the DMA cycles, from 2C00 to 2C40, and nothing
else
```

Tracepoints appear in `BREAK`'s listing with the rest, and their `hits` count the times they fired. A tracepoint never hides an ordinary breakpoint at the same address: if both are set, the trace flips *and* the machine stops.

A debugging session

The machine is not booting. Where does it get to?

```
altairsim> BREAK 2C00           the loader relocates itself to 2C00; does it get there?
altairsim> RUN FF00
breakpoint at 2C00
altairsim> REGS
altairsim> DISASM               what is it about to do?
altairsim> STEP 20              walk into it
```

The disk is not being read. Is the card even being asked?

```
altairsim> BREAK IO R 08        stop on any read of the controller's status port
altairsim> RUN FF00
```

Something is scribbling on the BIOS.

```
altairsim> BREAK MEM W E400
altairsim> RUN
```

The tools that are not here yet

`SNAPSHOT`, `RESTORE`, `RECORD` and `REPLAY` all **resolve** and all tell you they are waiting on the debugger. There is no rewind: `TRACE` logs the machine as it runs and `HISTORY` shows you the run-up to a stop, but you cannot save the machine's state and step *backwards* into it, and there is

no record-and-replay to reproduce a run from the start. When you stop the machine, you stop it where it is.

Say so plainly rather than let you find out: if you need to catch a bug that happens once in ten million instructions and you cannot predict where, a conditional breakpoint (`BREAK <addr> IF …`) and the `HISTORY` ring are the tools you have — but nothing here will replay it for you.

Symbols go one way for now: you can name an address (`BREAK START`), but `DISASM` and `DUMP` still print the machine in hex — they do not yet annotate a `JMP 0100` as `JMP START`. Loading the symbols is what that display will read when it lands.

Machines

A **machine** is a backplane with cards in it. Which cards, with what settings, in what state — that is all a machine is, and it is the only thing `altairsim` needs to be told.

You tell it in one of three ways: name a **built-in**, name a **file**, or say **nothing** and take the default. This chapter is about how that choice is made, and about the one rule that decides what a path means.

The command line

```
altairsim [options] [machine]
```

Option	
`machine`	a built-in name, **or** a config file if it contains a `/` or ends in `.toml`
`-m, --machine <name>`	**always** a built-in name — never a file
`-f, --file <path>`	**always** a file — never a built-in name
`-n, --none`	an empty backplane. No boards, no memory, nothing
`-l, --list`	list the built-in machines and exit
`-s, --script <file>`	run a command script, then exit with its status
`-x, --exec <cmd>`	run one monitor command, then exit. Repeatable
`-i, --interactive`	after `--script` / `--exec`, stay in the monitor
`--mcp`	run as an MCP server on stdio
`-v, --version` `-h, --help`	

Give exactly one machine. A positional name *and* a `-m`, or a `-f` *and* a `-n`, is an error — the program says *give ONE machine* and stops. It does not guess which one you meant.

How a bare word resolves — and why it never looks at your disk

```
$ altairsim basic4k                    a BUILT-IN, by name
$ altairsim disks/cpm22/cpm22-buffered.toml            a FILE, by path
```

The rule is **purely syntactic**. The filesystem is **never probed**:

The word	What it is
contains `/` or `\`	a **file**
ends in `.toml`	a **file**
anything else	a **built-in name**

That is deliberate, and it is worth being clear about why, because a simulator that guessed would be more convenient exactly until the day it was not.

`altairsim basic4k` means `basic4k` in every directory on earth. It means the same thing on your machine and on mine. It cannot be hijacked by a file called `basic4k` that happens to be lying next to you — because the program never asks whether such a file exists. A command in a script, a line in a README, a habit in your fingers: all of them keep meaning what they meant.

The cost is that `altairsim mymachine.toml` needs the extension, and `altairsim ./mymachine` needs the `./`. That is a small price, and if you want the question settled explicitly, settle it:

```
$ altairsim -m basic4k                 a built-in. Stop looking for a file
$ altairsim -f ./basic4k               a file called `basic4k`, no extension, right here
```

`-m` and `-f` are how you say what you mean when the syntax will not say it for you.

The one file the simulator *finds*

If you name **nothing at all**, and the working directory contains a file called `altairsim.toml`, that machine is loaded — and it says so:

```
$ altairsim
altairsim 0.1.0 -- 8080, full speed.
machine: ./altairsim.toml
altairsim>
```

This is the **only** file the simulator finds rather than is given, and it only happens when the command line names nothing whatsoever. Name a built-in, a file, or `-n`, and `./altairsim.toml` is ignored — you asked for something, so you get it.

Put one in a project directory and `altairsim`, bare, is your machine. **It announces itself when it does**, so you are never running something you did not know about.

With no `altairsim.toml` and no arguments, you get the built-in `default`.

The built-in machines

A **built-in is a TOML machine file compiled into the binary.** There is nothing privileged about it: same format, same keys, same rules as one you write yourself. It is in the program only so that it is always there.

```
$ altairsim --list
```

names them. They are `default`, `4k`, `altmon`, `basic4k`, `basic8k`, `ps2`, `ps2int`, `minidisk` and `z80`; the machine reference at the back of this manual says what each one is.

To see what is in one:

```
$ altairsim -x 'SHOW MACHINE' basic4k
```

And to get it as **text you can edit** — the actual machine file, every board, every setting:

```
$ altairsim -x 'CONFIG SAVE mine.toml' basic4k
$ altairsim mine.toml
```

`CONFIG SAVE` writes the machine you are actually running, and it round-trips. **Which makes every built-in a worked example.** Find the one closest to what you want, save it out, and edit it.

Or better, do not copy it at all — start *from* it with `base`, and write down only what is different. The configuring chapter is about that.

The empty backplane — `-n`

```
$ altairsim -n
```

No boards. No memory. No processor. `-n` is a bare chassis, and every `BOARDS ADD` from there is yours. It is the honest starting point when you are building a machine up card by card, and it is the one way to be certain nothing is in there that you did not put there.

The path rule, and it has two halves

This is the rule that lets an example directory be copied anywhere and still boot. It is one principle stated twice:

> **A path written inside a machine file is relative to that file.**
>
> **A path you type at the prompt is relative to your shell.**

Both are true at once, and they do not conflict, because they are the same idea: **a path means what its author could see.**

The author of a machine file could see the directory the machine file is in. When `disks/cpm22/cpm22-buffered.toml` says `mount = "cpm22b23-56k.dsk"`, it means *the disk lying next to me* — and it goes on meaning that after you copy the folder to your desktop, rename it, or mail it to someone. That is why the examples are self-contained directories, and why the quick start's `cp -R` actually works.

You, at the prompt, can see your own working directory. When you type

```
altairsim> MOUNT dsk0:drive1 scratch.dsk
```

you mean *the file next to me*, because that is the only thing `scratch.dsk` could sensibly mean when you are the one typing it. The simulator does not go rummaging in the machine file's directory for a file you named with your own hands.

The corollary is worth stating, because it catches people: **a path in a `startup` command is inside the machine file, so it is relative to the machine file** — even though `startup` commands look exactly like things you type. They were written by the file's author, so they mean what the file's author could see.

When it bites, and what it looks like

The rule is invisible until a file is missing, and then it can look like a typo that is not one. Keep your machine files in a `machines/` folder, write a path meaning *the folder you launched from*, and you get the one confusing case:

```toml
[[board.drive]]
unit  = 0
mount = "disks/Kermit/cpm.dsk"      # meant: the disks/ I can see from my shell
```

`altairsim -f ./machines/8800c.toml` then says:

```
./machines/8800c.toml: dsk0: 'machines/disks/Kermit/cpm.dsk': no such file
  ('disks/Kermit/cpm.dsk' is relative to the machine file that wrote it, in ./machines/)
```

The disk is not missing. It was looked for beside the machine file, because that is where the rule says a machine file's paths point. Write it the way the machine file sees it:

```
mount = "../disks/Kermit/cpm.dsk"   # up out of machines/, then down into disks/
```

…or keep the machine file next to what it mounts, which is what every shipped example does.

Note what is *not* affected: once the machine is up, `MOUNT dsk0:drive1 disks/Kermit/cpm.dsk` typed at the prompt needs no `../`, because you are the one typing it. The `../` belongs to the file, not to you.

Ask the machine, rather than working it out

You do not have to hold this in your head. `SHOW PATHS` prints every base at once, for the machine you are actually running:

```
altairsim> SHOW PATHS
  what you type      /home/you/altair
                     MOUNT, LOAD, SAVE and -s scripts resolve against this.

  machine file       /home/you/altair/disks/cpm22
                     `mount`, `base` and the MOUNT/LOAD lines in `startup`
                     resolve against THIS, not the cwd -- so a machine file
                     names the disks lying beside it and goes on naming them
                     from wherever you launch it.

  hb0 sandbox        /home/you/altair/disks/cpm22/xfer
                     THE GUEST'S SANDBOX, and the only real fence here:
                     R.COM/W.COM cannot leave it. It is not a base for
                     anything you type. Set with `hostdir`.
```

Three lines, three different directories, and they are allowed to differ — that is the rule working, not a fault.

Boot a **built-in** machine and the middle line says so, because then there is no file and nothing was re-based:

```
  machine file       (none -- this machine is built in to the binary)
```

`SHOW MOUNTS` is the companion: every disk, tape and ROM in the machine and what is in each, across all the cards at once.

```
altairsim> SHOW MOUNTS
  UNIT         KIND  HOLDS
  dsk0:drive0  disk  cpm22b23-56k.dsk
  dsk0:drive1  disk  (empty)
  mem0:rom0    rom   builtin:dbl  (read-only)
```

Those paths are printed **as written**, which is why the two commands belong together: `SHOW MOUNTS` tells you what the machine was told, and `SHOW PATHS` tells you what that meant.

None of this is a sandbox

The path rule decides **where a path points**, and confines nothing. A machine file may mount any file on your disk — with `..`, or with an absolute path — and it will be opened.

The one real fence is the Host Bridge's **`hostdir`**, which limits how far a CP/M program running *inside* the machine can reach when it reads and writes host files. That is a different mechanism for a different purpose — see *Moving files in and out* — and nothing you write in a machine file moves it.

Running a command and leaving — `-x` and `-s`

`altairsim` does not have to be interactive.

```
$ altairsim -x 'SHOW MACHINE' default
$ altairsim -x 'DUMP 0 F' disks/cpm22/cpm22-buffered.toml
```

`-x` runs one monitor command against the machine and exits. It is **repeatable**, and the commands run in the order you gave them:

```
$ altairsim -x 'MOUNT dsk0 mine.dsk' -x 'RUN FF00' -i disks/cpm22/cpm22-buffered.toml
```

`-i` is the difference between a query and a start-up: without it the program exits when the commands are done; with it you are dropped into the monitor with the machine exactly as your commands left it. `-i` alone, with no `-x` or `-s`, does nothing.

`-s` runs a **script** — a file of monitor commands, one per line, the same ones you type:

```
$ altairsim -s boot.cmd disks/cpm22/cpm22-buffered.toml
```

The exit status is non-zero if any command failed. That is the whole point: `altairsim -s` is a program you can put in a shell script, a Makefile, or a build, and test the result of.

```
if altairsim -s check.cmd mine.toml; then
    echo "machine is sane"
fi
```

Which chapter next

The **configuring** chapter is the machine file itself: every table, every key, and the four things a `[[board]]` entry can mean. The **boards** chapter is what the eleven cards *are*.

The machine file

A machine file is **TOML**. It lists the cards in the backplane, what is set on each of them, and what to do once the power is on. That is all it does, and there is nothing else it can do.

This chapter is the normative description of the format.

The one thing to know first

Anything you can type, a config can do — and nothing more. A machine file has no special powers. It cannot boot a disk, because there is no `BOOT` verb; what it can do is *type* `RUN FF00` *for you*, which is what the operator did. Every board setting in a machine file is a setting you could have made with `SET` at the prompt, and every `SET` you make at the prompt is a key you could have written in the file. **A board's properties *are* its TOML keys.**

There is no separate config schema anywhere in the program. That is why the board reference at the back of this manual is exhaustive, and why it cannot drift out of date: it is printed from the same table the monitor resolves against.

Nothing is silently ignored

Any unknown table or key is a hard error, with a sentence saying so. The machine does not load.

```
mine.toml: unknown [machine] key 'widget'
mine.toml: [[board]] cpu0: cpu0 has no property 'frobnicate'. Known: clock_hz idle
```

This is not pedantry. **A configuration that looks like it set something and did not is worse than one that will not load**, because you will spend the afternoon debugging the machine instead of the typo. A misconfiguration in this program cannot be silent.

The tables

Table	What it is
`[machine]`	the machine's identity. Three keys, no more
`[[board]]`	a card. One entry per card
`[board.unit.<name>]`	one unit *on* the card above — a serial channel, a tape deck
`[[board.region]]`	a memory region on a `memory` card
`[[board.drive]]`	a drive on a disk controller
`[console]`	**your terminal.** Not a board — see below

`[machine]` — and it has exactly three keys

```toml
[machine]
name    = "cpm22"
base    = "default"
startup = ["RUN FF00"]
```

`name`

What the machine is called. That is the whole of it.

`base` — start from a machine, and say what is *different*

```toml
base = "default"          # a built-in
base = "../cpm22/cpm22.toml"   # or a file
```

The value resolves by **the same syntactic rule as the command line**: contains a `/` or ends in `.toml` → a file; otherwise → a built-in name. (The machines chapter explains why the filesystem is never probed.) A file path is relative to *this* file.

Two rules about where it goes:

- **`base` is processed before every other key**, whatever order the file is written in.
- **`base` must appear before the first `[[board]]`.** You cannot inherit a backplane you have already started modifying.

`base` nests **up to 8 deep**. A machine built on a machine built on `default` is fine.

This is the key that makes the format worth using. Without it, every variant of a machine is a copy of a hundred lines, and the day you change one of them you change it in six files. With it, a machine file says only **what is different**, and reads as the diff it actually is.

`startup` — the operator's keystrokes, written down

```
startup = ["RUN FF00"]
```

An array of **ordinary monitor commands**, run once the machine is built. Any command. They run in order, and you see them run — that is the `startup>` line in the quick start.

Paths inside a `startup` command are relative to the machine file, not to your shell, because the file's author wrote them and the file's author could see the file's directory. This is the one place the two halves of the path rule sit next to each other, and it is worth remembering.

What `[machine]` will *not* take

```
[machine]
clock_hz = 2000000        # ERROR
sense    = 0x80           # ERROR
```

Both are **rejected, with an explanation**:

```
mine.toml: clock_hz belongs to the CPU CARD, not to [machine] --
  the crystal is on the card. Put it in the CPU's [[board]]:
      [[board]]
      type     = "8080"
      id       = "cpu0"
      clock_hz = 2000000
```

The crystal is soldered to the **88-CPU card**. The sense switches are on the **front panel**. Neither is a property of "the machine" — the machine is just the box they are plugged into. If you pull the CPU card out, the crystal goes with it.

These two get a bespoke error rather than the generic *unknown key* because they are the two people reach for first, and being told *where the thing actually lives* is more use than being told it isn't here.

`[[board]]` — and it has four forms

This is the heart of the format. **What a `[[board]]` entry means depends on whether it has a `type`, and whether its `id` is one the base already used.**

Write	And it means
`type` + a **new** `id`	**ADD** the card
`type` + an id **from the base**	**REPLACE** the card outright
no `type` + an id	**MODIFY IN PLACE**
`remove = true` + an id	**PULL THE CARD OUT**

`id` is always mandatory. It is how you refer to the card at the prompt, and how a later file refers to it here.

ADD — `type` + a new id

```
[[board]]
type = "virtc"
id   = "vi0"
```

A card that was not there is now there. **In a file with no `base`, this is the only form** — there is nothing to modify, replace or remove.

REPLACE — `type` + an id the base already used

```
[[board]]
type = "2sio"
id   = "sio0"
port = 0x20
```

If the base had a card called `sio0`, it is **gone** — pulled out and thrown away — and a fresh `2sio` is fitted in its place. **Everything the base set on that card is lost**, including the settings you did not mention. You get the type's defaults, plus whatever you write here.

That is what "replace" means, and it is almost never what you want. You want:

MODIFY IN PLACE — no `type`

```
[[board]]
id   = "cpu0"
clock_hz = 2000000
```

Leave the `type` out and you are reaching into the card that is already there. Everything the base set on `cpu0` stays set; you change the crystal and nothing else.

The absence of `type` is the whole signal. It reads oddly for about a day and then reads as exactly what it is: *I am not fitting a card, I am adjusting one.*

REMOVE — `remove = true`

```
[[board]]
id     = "acr0"
remove = true
```

The card is pulled out of the backplane. Its ports stop being decoded. Nothing else in the file may mention it.

The error that catches a copy-paste

`type` + an id that *this same file* has already declared is an error. Not a replace — an error. Within one file, declaring the same card twice is never something you meant; it is a block you copied and forgot to rename. The file will not load, and it will tell you which id.

(Across files it is different: an id from your *base* is a card you inherited, and replacing it is a legitimate thing to want.)

Everything else on a `[[board]]` is a property

`type`, `id` and `remove` are the only keys the config layer understands. **Every other key is handed straight to the board.**

```
[[board]]
type = "2sio"
id   = "sio0"
port = 0x10          # the 2sio knows what a port is. The config layer does not.
```

The config layer knows nothing about ports, baud rates, sense switches or drive counts. It cannot, and it does not try. It routes the key to the card and the card accepts it or rejects it by name:

```
mine.toml: [[board]] cpu0: cpu0 has no property 'frobnicate'. Known: clock_hz idle
```

The full key list for every card is the board reference at the back of this manual. The boards chapter says what the cards *are*.

`[board.unit.<name>]` — settings that belong to one unit

Some cards carry more than one independent thing. An 88-2SIO is **two 6850 ACIAs**, not one chip with two channels: unit `a` and unit `b` have their own baud rate, their own interrupt strap, their own endpoint, and they share nothing at all. So they get their own tables.

```
[[board]]
type = "2sio"
id   = "sio0"
port = 0x10                    # the CARD's property -- both chips live at this base

  [board.unit.a]
  baud    = 9600               # channel A's property
  connect = "console"

  [board.unit.b]
  baud    = 1200               # channel B is a different chip. It does not care.
  connect = "tcp:2323"
```

A key in `[board.unit.a]` is exactly the key `SET sio0:a baud=9600` takes at the prompt. It is the same property, reached two ways.

The board reference lists which cards have units, and what each unit takes.

`[[board.region]]` — memory

A `memory` card is **a list of regions**, which is why one physical card can carry 56K of RAM and a boot PROM at the top of memory. The regions are the card.

```
[[board]]
type = "memory"
id   = "mem0"

  [[board.region]]
  type = "ram"
  at   = 0x0000            # HEX -- it is an address
  size = "56K"             # DECIMAL -- it is a count

  [[board.region]]
  type  = "rom"
  at    = 0xFF00
  size  = 256
  mount = "turnmon.bin"    # relative to THIS FILE
```

Key	
`type`	**required.** `ram` or `rom`
`at`	the address it decodes. **Hex**
`size`	how big. **Decimal**; `K` and `M` suffixes work
`mount`	a ROM image: a file path, or `builtin:<name>`

(A size with a suffix is written as a string — `size = "56K"` — because `56K` is not a number TOML will accept bare. A plain count needs no quotes: `size = 256`.)

An empty socket

A `rom` region with no `mount` is an empty socket. It decodes nothing, and reads there float to `FF` — because that is what an S-100 bus with nobody driving it does. It is not zeros, and it is not an error. It is an unpopulated socket on a card that has one, which is a thing a real machine could be, and software that reads it gets `FF`.

`[[board.drive]]` — disks

A disk controller addresses drives; a drive holds an image.

```
[[board]]
id = "dsk0"

  [[board.drive]]
  unit     = 0            # DECIMAL -- it is a drive number
  mount    = "cpm.dsk"    # relative to THIS FILE
  readonly = false
```

Key	
`unit`	the drive number. **Decimal**
`mount`	the image file
`readonly`	refuse every write at the controller, so the host file cannot change. For a disk you mean to read — see the disks chapter
`media`	force a format instead of probing the image

`media` is the escape hatch. The controller normally works out the format from the image, and normally it is right; when it is not — a headerless image, an unusual geometry — you say so. The disks chapter covers the formats.

Numbers: one rule, and it is not negotiable

> **On the wire → HEX. Never on the wire → DECIMAL.**

An address, a port, a data byte, a sense-switch setting is something the 8080 sees on the bus. It is written **hex**, and it is written hex *without a prefix*:

```
port  = 10        # 0x10. SIXTEEN. This is the 2SIO's default port.
at    = 0xFF00    # an address
sense = 80        # 0x80
```

A count, a size, a baud rate, a drive number is a **quantity**. The 8080 never sees it. It is written **decimal**:

```
baud   = 9600     # nine thousand six hundred
drives = 4        # four
size   = 56       # fifty-six bytes
unit   = 0
```

`port = 10` is port sixteen. Read that line again, because it is the one that will get you, and it is the one the rule exists to make predictable. A port is on the wire. Ports are hex. The 88-2SIO lives at 10 hex and every listing from 1976 writes it that way.

If you want to be explicit — and in your own files, be explicit — say so:

`0x10`, `$10`, `10h`	**hex**, whatever the key
`#16`	**decimal**, whatever the key
`56K`, `1M`	always **decimal**. A suffix implies a count

The board reference prints every default **in its own base**, so there is never a question about which one a given key is.

`[console]` — your terminal, which is not a board

```
[console]
attn      = 0x05      # the key that gets you back to the monitor. ^E
upper     = false
strip7in  = false
strip7out = false
crlf      = false
echo      = false
bell      = true
bsdel     = "off"
```

`[console]` is **not a `[[board]]`** and it is not in the backplane. It describes *the terminal you are sitting at* — a piece of equipment on your desk, on the far end of a cable, in 2026. The Altair never knew anything about it.

Key	
`attn`	the escape byte. **Hex.** Default `05` = `^E`
`upper`	fold input to upper case
`strip7in`	clear bit 7 of everything the guest receives
`strip7out`	clear bit 7 of everything the guest sends
`crlf`	translate line endings
`echo`	echo typed characters locally
`bell`	let the guest ring your terminal's bell
`bsdel`	`off` | `bs` | `del` — what your Backspace key sends

The transform chain belongs to the console, and only to the console

This section is here because the temptation to solve it in the wrong place is very strong, and solving it in the wrong place silently corrupts your data.

MITS BASIC sets bit 7 of the last character of every message, as a string terminator. That is not a bug. It is how the interpreter marks the end of a prompt. Send all eight bits to a modern terminal and every prompt in the program arrives wearing garbage:

```
MEMORY SIZ?
```

The fix is **`strip7out` on the `[console]`**:

```
[console]
strip7out = true
```

And the reason that is the *right* fix — rather than a plausible one — is that it is what actually happened. **The real machine sent all eight bits.** The 88-SIO put the byte on the wire exactly as BASIC handed it over. And the **Teletype ignored the eighth**, because a Model 33 is a 7-bit terminal and always was. The stripping was done by the terminal, at the far end, in 1976. So it is done by the terminal here.

Now the two wrong fixes, because they both look reasonable and they are both traps:

- **It is not a 7-bit strap on the card.** You could set `data_bits = 7` on the SIO and the prompt would come out clean. You would also have quietly made that port unable to carry a binary byte — and the day you run XMODEM through it, every byte with the top bit set is mangled. **Line coding is a *frame*, not a *mask*.** It is real hardware and it is modelled, but it is not this.
- **It is not a filter on the line.** Same failure, one layer up, and just as silent.

Every serial line in this simulator is 8-bit clean, because a line carries XMODEM, and a line that eats bit 7 is a line you cannot trust with a file. The transforms — `strip7out`, `upper`, `crlf` and the rest — are properties of the **console**, and the console is the one place in the system where mangling bytes for a human's benefit is the correct thing to do.

A complete machine file

Small, whole, and it works. No `base` — so every board is an ADD:

```toml
[machine]
name    = "tiny"
startup = ["RUN 0"]

[[board]]
type = "fp"               # the front panel: sense switches and lamps
id   = "fp0"
sense = 0x00

[[board]]
type     = "8080"         # the CPU is a card. The crystal is on it.
id       = "cpu0"
clock_hz = 0              # 0 = flat out. This is the default.

[[board]]
type = "2sio"             # the console card
id   = "sio0"
port = 10                 # HEX. Port SIXTEEN.

  [board.unit.a]
  baud    = 9600          # DECIMAL. Nine thousand six hundred.
  connect = "console"

[[board]]
type = "memory"
id   = "mem0"

  [[board.region]]
  type = "ram"
  at   = 0x0000
  size = "16K"

[console]
strip7out = true
```

A `base` delta — and this is the one you will actually write

This is genuine. It is how the CP/M example is built, and it is nine lines:

```toml
[machine]
name    = "cpm22"
base    = "default"        # a front panel, an 8080, a 2SIO console, a floppy
                           # controller, 56K of RAM and the boot PROM at FF00
startup = ["RUN FF00"]     # the operator's own keystrokes, written down

[[board]]
id = "dsk0"                # NO type: modify the controller the base already has

  [[board.drive]]
  unit  = 0
  mount = "cpm.dsk"        # relative to THIS FILE
```

Everything a 56K CP/M machine is, `default` already was. The only thing this file has to say is *which floppy is in drive 0*, and *press RUN at FF00*. **That is the whole of the difference, and so that is the whole of the file.**

Note what is not here: no `type` on the `[[board]]`, because `dsk0` already exists and we are adjusting it, not fitting it. Had we written `type = "dcdd"`, we would have thrown the base's controller away and got a fresh one with default settings — and it would still have worked, and we would never have known we had done it. Leave the `type` out.

Saving and loading at the prompt

```
altairsim> CONFIG SAVE mine.toml
altairsim> CONFIG LOAD mine.toml
```

`CONFIG SAVE` writes the machine you are actually running — every card, every property, as it stands right now, including everything you changed with `SET` since you started. It **round-trips**: load what it wrote and you get the machine back.

`CONFIG LOAD` is the whole machine, so it replaces the one you have — the same thing that naming the file on the command line does, and there is no undo but the file you saved it to. It is also **all or nothing**: the machine is built off to one side first, so a file that will not load leaves you exactly where you were rather than halfway between two machines.

Which makes it the fastest way to write a machine file. Build the machine at the prompt with `BOARDS ADD` and `SET` until it is what you want, then save it, then edit the file down to the parts you care about — or give it a `base` and delete the rest.

Boards

An Altair is a **backplane**. Everything else is a card in it — the memory, the serial ports, the disk controller, the front panel, and the processor itself. There is no "machine" underneath the cards doing the real work; take the cards out and there is nothing left but a bus.

`altairsim` is built that way on purpose, and it is the reason the CPU's crystal is a property of the CPU *card* and the sense switches are a property of the *front panel*. Not pedantry: it is what lets you pull a card out, put a different one in, and find out what the software does about it.

This chapter says what the eleven cards **are** — what the real hardware was, what it is for, and what will bite you. **It does not list their parameters.** Every key of every card is in the board reference at the back of this manual, printed from the program's own tables, which is why it cannot be wrong.

The eleven cards

Type	What it is
`memory`	RAM and ROM, as a list of regions
`8080`	the MITS 88-CPU
`z80`	a Z80 CPU card — the same bus, a different instruction set
`2sio`	MITS 88-2SIO — two serial ports. The usual console
`sio`	MITS 88-SIO — one serial port. MITS's first
`acr`	MITS 88-ACR — the cassette interface
`dcdd`	MITS 88-DCDD — the 8″ floppy controller
`mds`	MITS 88-MDS — the 5¼″ minidisk controller
`virtc`	MITS 88-VI/RTC — vectored interrupts and a clock
`fp`	the front panel
`hostbridge`	file transfer to your host. **Ours, not a period card**

`memory` — RAM and ROM

A memory card is **a list of regions**, and the regions are the card. That is not a modelling convenience; it is what an S-100 memory board was. One physical card carried banks of chips

decoding whatever ranges its jumpers said, and a card with 56K of RAM low and a 256-byte boot PROM at `FF00` is a perfectly ordinary card.

So `default` has exactly one memory board in it, and that board is the 56K *and* the PROM.

`PHANTOM*` — how a boot PROM gets out of the way

The bus has a line called `PHANTOM*`. **A board pulls it to switch another board off.** When the PROM at `FF00` is being read, it asserts `PHANTOM*`, and the RAM card underneath — if it is jumpered to honour it — shuts up. Two cards decode `FF00`; only one answers.

This is how a disk Altair boots. The PROM overlays the RAM at the top of memory, the loader runs out of it, and then **the loader gets out of the way** and the RAM underneath is uncovered — which matters, because CP/M wants that memory back.

Whether a card honours `PHANTOM*`, and whether it asserts it, are **jumpers**. They are on the card and they are yours to set. Getting them wrong produces a machine that does not boot and does not say why — which is precisely what it did in 1977, and the bus view in the monitor will show you both cards claiming the page.

Banking

Sixty-four kilobytes was not enough for very long, and the industry's answer was **bank switching**: several cards' worth of RAM at the same addresses, with a port that says which one is live. Nobody agreed on how.

`altairsim` implements five of the real schemes — **ExpandoRAM**, **Vector Graphic**, **Cromemco**, **North Star Horizon**, and **AB Digital B810** — and **no two are alike**. Different ports, different bit meanings, different numbers of banks. That is not a failure of the simulator to generalise; it is the actual history, and software written for one will not drive another.

If you are not running banked software, leave banking off. It is off by default.

`8080` — the MITS 88-CPU

The processor is a card like any other. It plugs into the backplane, it can be removed, and with `-n` you can build a machine that does not have one.

It decodes no ports and answers no addresses. What it does is **drive the bus** — which makes it unlike every other card in the box, and is exactly what the 88-CPU did.

The crystal is on the card

Which is why **`clock_hz` is this card's property and not the machine's** — and why writing it in `[machine]` is an error with an explanation rather than a setting that quietly does nothing.

`clock_hz = 0` is the default, and it means run flat out — as fast as your host can go. On a modern machine that is somewhere north of a hundred times a real Altair.

```
SET cpu0 clock_hz=2000000
```

buys back the real 2 MHz machine, and it is worth doing at least once. **What the guest *sees* is identical either way.** Instructions cost the right number of T-states, a cassette takes the right number of them to load, and the disk turns at the right speed regardless. The crystal buys period ***feel***, not period ***behaviour***. Watch BASIC print its banner at 2 MHz and you learn something about 1976 that no amount of reading will teach you. Then set it back.

With one boundary, and it is a real one: that guarantee ends at the edge of the machine. Everything *inside* keeps time by the same T-states, so it all agrees with itself at any speed. But a guest program counts instructions to measure a second — `PCGET`'s timeout is a 49-T-state loop spun 159 × 256 times — and flat out, your host retires its "three seconds" in a few tens of milliseconds while the program at the other end of the wire is still using real ones. **Anything the guest times against the outside world wants the real crystal**: XMODEM through a serial port is the case you will meet. See the troubleshooting chapter.

`idle` — the CPU stands down at a prompt

A guest sitting at a prompt is not doing anything. It is spinning on the serial card's status register, waiting for a byte that has not arrived, and at `clock_hz = 0` it will spin as fast as your CPU can let it — one core, pinned, indefinitely, to accomplish nothing.

`idle` (on by default) stands the processor down when the guest is only polling an empty keyboard. A hundred percent of a core becomes about three and a half.

The guest cannot tell. Not "the guest probably won't notice" — it *cannot tell*, because the moment a byte arrives the processor is back before the guest's next poll could have seen anything different. An XMODEM transfer through an idling machine is byte-exact.

`z80` — a Z80 CPU

A second processor card. It plugs into the same backplane as the 88-CPU, decodes nothing, and drives the bus the same way — the only difference is the instruction set behind it. Put a

`z80` where an `8080` would go and the bus, the boards, and the debugger neither know nor care; that is the whole point of keeping the processor a card.

It carries the same three properties as the 8080 — `clock_hz` (the crystal, flat out by default), `idle` (stands down at a prompt), and the read-only `achieved_hz` — and each means exactly what it means there.

The core is validated the same way the 8080 was, against the same kind of gate: ZEXDOC and ZEXALL, the standard Z80 exercisers, both pass before a single board is built on top of it. The built-in `z80` machine is a minimal one — a `z80`, 64K of RAM, and a 2SIO console — for putting it through its paces.

`2sio` — MITS 88-2SIO

Two **6850 ACIAs**, units `a` and `b`, four ports at BASE+0 through BASE+3. Base defaults to `10` hex, which is where every listing from the period expects it.

This is **the usual console card**, and it is what `default` has. If you are running CP/M or Microsoft BASIC, this is the card the software is talking to.

The two halves share nothing

Not the baud rate, not the endpoint, not the interrupt strap. **They are two independent chips that happen to be bolted to the same board**, and the model says so: `a` and `b` are separate units with separate properties.

So a console on `a` at 9600 and a modem on `b` at 1200, one interrupting and one polled, is not a configuration you have to work around. It is Tuesday.

The serial chapter covers what a channel can be *connected* to: your terminal, a TCP socket, or a real serial port on your host, with the modem control lines wired through.

`sio` — MITS 88-SIO

One **COM2502 UART**, unit `tty`, two ports. This was **MITS's first serial card** — it predates the 2SIO, and the earliest Altair software talks to it. `basic4k` uses it.

Its status bits are inverted

A clear bit means ready. Read that twice, because every instinct you have says otherwise, and because it will make you certain you have found a bug in the simulator.

You have not. **It is a fact about the chip**, not a quirk anyone invented: the COM2502's status lines came out of the package active-low, MITS wired them to the data bus as they were, and every program that drove an 88-SIO was written knowing it. `basic4k`'s I/O routine masks and branches on zero, and it is right to.

The port must be **even**: control at BASE, data at BASE+1.

`acr` — MITS 88-ACR

The **cassette interface**: an 88-SIO channel B with an FSK modem on the end of it, so a byte on the bus becomes an audible tone on a tape and back again. Unit `tape`, default port `06`, and it runs at 300 baud because that is what an audio cassette could carry.

It brings its own verb — `REWIND` — because a tape has a position and a disk does not, and pretending otherwise would help nobody.

This is the card that shows you what an Altair actually was: no disk, no PROM, a bootstrap you toggle in by hand, and eight minutes of listening to a cassette. **The tapes chapter is the one to read**, and Altair 4K BASIC 3.1 is the machine to run.

`dcdd` — MITS 88-DCDD

The **8″ hard-sector floppy controller**, up to sixteen drives, three ports at `08`, `09` and `0A`. **This is the card CP/M booted from**, and it is in `default`.

It also carries the **8 MB medium** — a large-capacity format the same controller can address.

Its status bits are **inverted**, for the same reason the 88-SIO's are and with the same consequence: a clear bit means ready.

The disks chapter is where this card lives: formats, mounting, write protection, and the track-buffer trap that means you should get back to the `A>` prompt before you stop the machine.

`mds` — MITS 88-MDS

The **5¼″ minidisk**, four drives. **The same registers as the DCDD** — a program written for one will drive the other — but **different physics**:

	`dcdd`	`mds`
Spindle	360 RPM	**300 RPM**
Byte time	32 µs	**64 µs**
Motor	always turning	**stops after 6.4 seconds**

The minidisk's motor is not permanently on. It spins up, and if nobody touches the drive it spins down again — which the software has to cope with, and which you can watch it cope with by setting the motor to `real`. By default the motor is `free`: always at speed, no waiting. The DCDD needs no such switch, because its spindle never stopped.

It cannot share a machine with a `dcdd`

Same three ports. Two controllers decoding `08` is not a limitation of this program; it is the MITS address map, and a real Altair with both cards in it would have had them fighting on the data bus. Fit both here and the bus view will name the contention rather than leave you wondering why the guest has gone strange.

Pick one.

`virtc` — MITS 88-VI/RTC

Two things on one card, which is why it has an awkward name.

Vectored interrupts. Eight lines, **VI0 through VI7**. A device is strapped to a line; when it interrupts, **level *n* becomes `RST n`** — the processor jumps to `8×n` and the right handler runs without anybody having to poll anything. **VI0 is the highest priority**, and the card enforces that: a lower level cannot interrupt a higher one that is being serviced.

This is what turns a machine that busy-waits into a machine that gets on with something else. Every card with an interrupt strap in its properties — the serial cards, the floppy controllers — is strapped to one of these lines, or to `int`, or to nothing at all.

A real-time clock, on the same board: a periodic interrupt off the 60 Hz line or off the system clock, divided down.

One port at `FE`, and it is **write-only**. There is nothing to read back. Interrupt cards are the easiest thing in a machine to get subtly wrong, and this one is worth reading the reference for before you strap anything to it.

`ps2int` is the machine that shows it working.

`fp` — the front panel

The switches and the lamps. The panel is **a card**, because on a real Altair it was one — it plugged into the bus like everything else, and a machine without it is a machine you cannot toggle a bootstrap into.

The SENSE switches, at port `FF`

The eight switches on the left of the address bank, `SA8`–`SA15`. **`IN FFH` reads them.** They are **read-only**: an `OUT FF` is not this card's business and goes nowhere at all.

They are not decoration. Period bootstraps read the sense switches to decide **what to boot from** — which device, at which port, at which speed. That is why every tape machine in this package sets one:

```
sense = 0x80        # basic4k: load from the 88-SIO at port 00
sense = 0x8E        # ps2:     the 2SIO, and interrupts off
```

Get it wrong and the loader sits there reading a device that is not there. It will not tell you. It has no way to.

`sense` is a **board property**, because the switches are on the panel. There is no machine-level `sense`, and asking for one is an error that says so.

`hostbridge` — ours, not a period card

This card never existed. It is the one thing in the machine that is not history, and the manual says so plainly rather than letting you discover it in a museum catalogue.

It moves **files between the guest and your host**, in both directions, and it is **sandboxed**: the guest sees one directory you nominate and **cannot escape it**. Not by `..`, not by an absolute path, not at all. That is a hard requirement, not a setting with a default.

Default port `B0`. **Nothing of the utilities is in the card.** `R`, `W` and `HDIR` are ordinary CP/M `.COM` programs — they live on a disk, they run at the `A>` prompt, and they talk to the card through its two ports exactly as any other CP/M program would. What is unusual is only where they came from: they were assembled *inside the machine*, by the machine's own assembler, from 8080 source that ships with it. So they are readable code rather than a magic trick the simulator performs on your behalf.

The file-transfer chapter is where this card is explained. It is the fastest way to get your own code into CP/M, and it beats XMODEM by a distance — but XMODEM works too, over an ordinary serial line, exactly as it did.

Working with the backplane at the prompt

Everything a machine file can do to a card, you can do by hand.

Command	
`BOARDS`	what is in the backplane
`BOARDS TYPES`	what you can add
`BOARDS ADD <type> <id>`	fit a card
`BOARDS REMOVE <id>`	pull one out
`SHOW <id>`	one card's settings, with the legal values
`SET <id> <key>=<value>`	change one

```
altairsim> BOARDS ADD virtc vi0
altairsim> SET vi0 rtc_source=line
altairsim> SHOW vi0
```

The keys are the same keys. `SET cpu0 clock_hz=2000000` at the prompt and `clock_hz = 2000000` in a machine file are the same property reached two ways — there is no separate config schema, which is the whole reason the board reference at the back can be exhaustive.

And when you have the machine you want:

```
altairsim> CONFIG SAVE mine.toml
```

writes it out, and it round-trips.

Contention

Two cards decoding the same port is **contention**, and it is a real thing that real backplanes did. Both cards answer, both drive the data bus, and what the processor reads is neither.

The simulator does not stop you. It does something more useful — **it tells you**:

```
altairsim> SHOW BUS CONTENTION
```

which names the port and names both cards. Fit an `mds` in a machine that already has a `dcdd` and this is what you will see, and it is a great deal more helpful than a guest that has mysteriously gone mad.

Being able to build a machine that does not work is not a defect. **It is the point** — this is a bench for developing hardware, and hardware that cannot be wired up wrong cannot be wired up at all.

Disks

A disk Altair is a floppy **controller** on the bus, some number of **drives** hanging off it, and a **disk image** sitting in one of those drives. The three are separate things, and the manual keeps them separate, because the machine did.

The controller is a board. It goes in the machine file. The drives are part of the controller — a MITS 88-DCDD addresses up to sixteen of them, whether or not you own sixteen. The disk goes in a drive, and you may put it there either way: name it in the machine file, or `MOUNT` it at the prompt. They do the same thing.

> **A disk is not a cassette, and the difference is real.** A machine file *may* name the floppy that is in drive 0 — the CP/M example does exactly that — but it may never name the tape in the recorder. A floppy drive is wired to its controller and the guest can tell what is in it. **A cassette recorder has no motor control on the card**: nothing in the machine can start the tape, sense it, or know it is there. So the tape is something a human puts in and presses PLAY on, and it stays at the prompt where humans are. See the tapes chapter.

The two controllers

Card	What it is	Drives	Ports
`dcdd`	**MITS 88-DCDD** — the 8″ hard-sector floppy controller. The one CP/M booted from.	up to 16	08, 09, 0A
`mds`	**88-MDS** — the 5¼″ minidisk. Smaller, slower, four drives.	4	08, 09, 0A

Read the ports column again. **They are the same ports**, which means the two cards cannot coexist in one machine. That is not a limitation of the simulator; it is the MITS address map. A real Altair with both would have had two cards decoding 08 and fighting on the data bus, and the bus view in the monitor will tell you so if you try it. Pick one.

Drives are units on the card: `drive0`, `drive1`, and so on up to the card's limit.

Putting a disk in — `MOUNT`

```
MOUNT <id>[:<unit>] <file> [RO]
```

```
altairsim> MOUNT dsk0:drive1 my-scratch.dsk
```

That is a floppy going into drive 1 of the controller called `dsk0`. The socket was empty before; it isn't now.

`WP` is **the write-protect tab**, and it does what the tab on a real diskette did: the guest may read the disk, and a write is refused at the controller and never reaches the file on your host.

```
altairsim> MOUNT dsk0:drive2 golden-master.dsk WP
```

`RO` is accepted and means exactly the same thing. It is the right word for a ROM socket, which has no tab to move — for a floppy, `WP` is the thing you are actually doing.

The guest is not told, and cannot be — see "Read-only is not an error message" below. Mount `WP` for a disk you intend to read.

A protected disk says so wherever it is listed, so you never have to remember which one you did it to:

```
altairsim> SHOW MOUNTS
  UNIT         KIND  HOLDS
  dsk0:drive2  disk  golden-master.dsk  (write-protected)
```

And if the **host** would not let us write the file — the permissions say no — the disk still mounts, protected, and says that it was not your idea:

```
  dsk0:drive2  disk  master.dsk  (write-protected -- THE HOST WON'T LET US WRITE IT; you did not
ask for this)
```

That one is worth reading closely. You did not ask for the tab, so CP/M is about to bounce every write off a disk you believe is writable.

And taking it out:

```
altairsim> UNMOUNT dsk0:drive1
```

The socket is empty again. The guest sees a drive with no disk in it, which is a thing a real drive could be.

Names, and what you may leave out

Board names are **case-blind everywhere** — `dsk0`, `DSK0` and `Dsk0` are the same card.

Beyond that you may omit **what carries no information**:

- the **trailing index**, when only one card of that type is in the machine. One floppy controller means `dsk` finds `dsk0`.
- the **unit**, when the card has only one thing you could possibly mount into. `MOUNT ACR tape.bin` needs no unit, because a cassette recorder has one slot.

A floppy controller does not qualify for the second one. It has sixteen drives, and which drive you meant is real information, so you must say it. **Anything genuinely plural, you must name.**

…or put it in the machine file

`MOUNT` at the prompt is for the disk you are dealing with *now*. A disk that belongs to a machine belongs in the machine file, and that is how the shipped examples do it:

```
[[board]]
id = "dsk0"                    # no `type`: the controller is already there

  [[board.drive]]
  unit  = 0
  mount = "cpm22b23-56k.dsk"   # relative to THIS FILE
  # readonly = true            # refuse every write; your file cannot change
```

The two forms do the same thing. The difference is only *when*: one is the drive as the machine ships, the other is you, at the prompt, changing your mind.

Note the path. It names the file lying **beside the machine file**, with no directory at all — which is what lets the whole folder be copied somewhere else and still boot. A path inside a machine file is resolved against **that file**; a path you type is resolved against **your shell**. The machines chapter has the rest of that rule.

The geometry is probed, not declared

You do not tell `altairsim` what kind of disk you just mounted. It **looks at the file's byte count** and works it out:

Format	Tracks	Sectors	Bytes/sector	File size
`8in`	77	32	137	**337,568**
`minidisk`	35	16	137	**76,720**
`fdc8mb`	2048	32	137	**8,978,432**

A file of 337,568 bytes is an 8″ floppy. There is nothing else it could be. This is why the quick start never mentions a format: there was nothing to mention.

When the size genuinely cannot decide — a truncated image, a format you are inventing — a `media` key on the drive forces the answer.

Why an 8 MB disk works at all

`fdc8mb` is a 2048-track disk on a controller that MITS designed for 77 tracks, and it works through the **stock card with the stock PROM**. That deserves an explanation, because it looks like a cheat and is not one.

The controller cannot tell an 8 MB disk from an 8″ floppy. It steps the head in, it steps the head out, it shifts bytes past a read head, and it reports what the drive tells it. It never asks how big the disk is, because a real 88-DCDD had no way to ask and no reason to want to. Step it 300 times and it steps 300 times. All the intelligence about *where track 1500 is* lives in the BIOS, which is software, and software can be rewritten.

The corollary is the useful part: **format and spindle are per drive, not per controller.** Mixed geometry on one card is the intended arrangement, not an accident — period 8 MB CP/M BIOSes expect exactly that, an 8 MB disk on A: and B: and ordinary 77-track floppies on C: and D:, so that you can `PIP` between the big disk and something you can hand to somebody else.

```
altairsim> MOUNT dsk0:drive0 big.dsk
altairsim> MOUNT dsk0:drive2 floppy.dsk
altairsim> SHOW dsk0
```

THE TRACK BUFFER TRAP

Read this section. It is the one thing about disks in this simulator that will cost you work if you do not know it, and it is not the simulator's doing — **it is what a BIOS may do**, and the CP/M in this package is one that does.

A track-buffering BIOS does not write to the controller when CP/M closes a file.

`BIOS WRITE` copies your data into a **track buffer in memory** — 32 sectors of 137 bytes, one whole track — marks it dirty, and returns. Nothing has touched the disk. Nothing will touch the disk until something calls the flush. And the flush is called from **`CONIN`**: the BIOS console-input routine. **Reading the console is the flush trigger.**

This is not madness. A track write is one seek and one revolution instead of thirty-two, and the moment the machine sits down to wait for a human to type something is precisely the moment

it has spare time and nothing to lose. It is a very good piece of engineering. It is also a loaded gun.

Not every CP/M does this

Buffering is not a property of CP/M, and it is not a property of the controller. It is a decision the author of a **BIOS** made, and the BIOS is precisely the part of CP/M that every machine's owner rewrote for himself. Period Altair BIOSes went both ways:

- **Track-buffered.** Writes pile up in memory and reach the disk on a console read. The CP/M that ships in this package is one of these.
- **Write-through.** Every `BIOS WRITE` puts a sector on the disk before it returns, and when you get your prompt back your file is already there. Some of these hook `CONIN` too — but only to *unload the head*, which is a courtesy to the drive and touches no data.

You cannot tell which one you have by looking at the `A>` prompt, and a disk image somebody handed you arrives with whatever BIOS its author wrote on it. So treat the rule below as the rule regardless: on a write-through BIOS it costs you nothing, and on a buffered one it is the difference between having your file and not.

> **Never `UNMOUNT`, copy, or kill the program immediately after a file operation. Get back to the `A>` prompt first.**

The `A>` prompt is CP/M reading the console; the console read flushes the buffer if there is one, and the directory update lands on the disk. Once you are looking at `A>`, the disk on your host is what you think it is — under either kind of BIOS. Save the file, watch the prompt come back, *then* do whatever you were going to do.

`^E` back to the monitor from the `A>` prompt is safe. `^E` back to the monitor one instant after `ED` says it has written your source file is **not**, and on the CP/M in this package the file will not be there.

The disk is real, and there is no undo

A mounted disk is mounted **read/write**, and every write goes through to the file on your host as it happens. There is no journal, no snapshot, and no way back. A CP/M cannot save your work onto a disk it is not allowed to write, so read/write is the only default that lets the machine be a machine — and the price of it is that you can destroy the example disk with one mistyped `ERA`.

Copy the directory before you experiment. It is self-contained and boots from anywhere.

Use `RO` when you want the guest to look and not touch.

Read-only is not an error message

`RO` is a promise to **you**, about your file. It is not a message to the guest, and this is worth being exact about, because it is easy to assume otherwise.

The controller has no way to say "write protected." The 88-DCDD's status byte is seven bits — write-circuit-wants-a-byte, head-movement-OK, head-loaded, drive-enabled, interrupts-enabled, on-track-0, new-read-data — and none of them means *the notch is covered*. A program reading that byte cannot distinguish a protected disk from an ordinary one.

And CP/M has nowhere to put the answer even if it had it. A CP/M 2.2 BIOS write returns `0` for success and `1` for a non-recoverable error. That is the entire vocabulary. There is no "protected" code, which is why a genuine hardware write failure surfaces as the blunt `Bdos Err On A: Bad Sector` — CP/M is telling you the only thing the interface let it hear.

The message people remember — `Bdos Err On A: R/O` — is a **different mechanism entirely**. That is CP/M's own software read-only flag, the one `STAT A:=R/O` sets and a warm boot clears. It lives in CP/M's head, not on the disk and not in the controller, and a write-protected image will never produce it.

So: a guest that tries to write to an `RO` disk is asking for something it cannot be refused politely. **Do not mount `RO` and then expect CP/M to cope gracefully** — mount `RO` for a disk you are going to read. If you want a disk the guest can write and you can throw away, copy the directory; that is what the copy is for.

Making a scratch disk

Two ways, and they arrive at the same place.

From the monitor. Mount a filename that does not exist yet:

```
altairsim> MOUNT dsk0:drive1 my-scratch.dsk
```

From inside CP/M. Boot, and format it the way the period did — the CP/M disk in the package carries a formatter, and formatting an image is formatting a disk. This is the more faithful route and it is the one that produces a disk the guest is certain to be happy with, because the guest made it.

Either way, remember which chapter you are in: get back to `A>` before you go looking at the file.

Looking at what is in the machine

`SHOW` on the controller lists its drives and what is in each one:

```
altairsim> SHOW dsk0
```

`BOARDS` shows the backplane — every card, where it decodes, and who is fighting whom:

```
altairsim> BOARDS
```

Between them they answer nearly every "why is it not booting" question there is, and the first one is almost always *the disk is in the wrong drive*.

Tapes

Before the floppy, there was a **cassette recorder**. Not a special one — a domestic audio cassette deck from a department store, with a microphone jack and an earphone jack, and MITS sold you a card that turned bytes into a noise it could record and turned the noise back into bytes. That card is the **88-ACR**, and it is in this simulator.

This chapter is how you load Altair 4K BASIC 3.1 the way it was actually loaded in 1976: nothing in ROM, nothing on a disk, a bootstrap you enter by hand, and a tape.

The card

`acr` is the **MITS 88-ACR** — an Audio Cassette Record/playback interface.

Underneath, it is **an 88-SIO channel B with an FSK modem soldered to it**. That is not a metaphor; that is what MITS built. The guest talks to a perfectly ordinary serial port, and the modem on the other side of it turns the bits into tones. Default port **06** (`0x06` status/control, `0x07` data). **300 baud**, which is the tape's speed, not a setting you picked.

It has one unit: **`tape`**. There is one slot in a cassette recorder.

`mode = play | record`

```
altairsim> SET acr0:tape mode=record
```

Play and record are **mutually exclusive**, and not because it was easier to write that way. It is **one head and one physical button**. You cannot press PLAY and RECORD at the same time on a cassette recorder, so you cannot do it here.

You cannot **`CONNECT`** it to anything

The ACR takes no endpoint. It cannot be wired to your terminal, to a socket, or to a real serial port, and the reason is the same one: **the line is soldered to the modem.** There is no connector on the back of an 88-ACR because there is nothing to connect — the signal goes to a cassette deck and nowhere else. **It is a cassette interface, not a serial port**, and the fact that it is built out of a serial port does not make it one. The serial chapter is about the cards that *do* take endpoints.

The tape is not hardware

The tape is NOT in the machine file, deliberately.

A machine file describes the **hardware** — what cards are in the backplane, what they decode, how much memory is on the bus. Which cassette is sitting in the recorder is not hardware. It is a thing you did with your hands, this morning, and you can do a different thing with your hands this afternoon without unscrewing the lid.

There is also a mechanical reason, and it is the honest one: **the card has no motor control.** An 88-ACR cannot start the tape, stop it, or rewind it. It can only listen to whatever is going past the head. The machine genuinely does not know a tape is there. So the simulator does not pretend that it does.

You put the tape in, and you press PLAY. That is `MOUNT`, and it is a thing you type.

Putting a tape in

```
altairsim> MOUNT acr0:tape "tapes/basic/4K BASIC Ver 3-1.tap"
```

The ACR has one unit, so the unit carries no information, so you may drop it:

```
altairsim> MOUNT ACR tape.bin
```

Names are case-blind. The rules are the ones in the disks chapter, and they are the same rules.

`REWIND`

The ACR brings a verb of its own:

```
REWIND <id>:tape
```

`REW` will do. It winds the cassette back to the beginning.

You need it to load the same tape twice. A tape that has been read is a tape whose head is at the end of the tape, and playing it again gets you silence. This surprises people exactly once.

```
altairsim> REW acr0:tape
```

Loading Altair 4K BASIC 3.1 — the three-step ritual

This is the whole point of the chapter. It is three commands, and it is what an Altair owner did every single time they wanted to use BASIC, because there was nowhere to keep it.

1. Put the cassette in.

```
altairsim> MOUNT acr0:tape "tapes/basic/4K BASIC Ver 3-1.tap"
```

2. Toggle in the bootstrap.

```
altairsim> LOAD "tapes/basic/LDR4K31.HEX"
```

About twenty bytes. **On a real Altair you entered this by hand on the front-panel switches** — eight switches, one byte, DEPOSIT, again, twenty times — and if you got a bit wrong you found out by the tape not loading. MITS printed the listing in the manual and expected you to type it in with your fingers. `LOAD` is doing exactly that job and no more: it is not a loader, it is a pair of hands.

3. Run it from address zero.

```
altairsim> RUN 0
```

The bootstrap starts the ACR reading, the tones come off the tape, and BASIC lands in memory and starts itself.

Then BASIC asks you the three questions:

```
MEMORY SIZE?
TERMINAL WIDTH?
WANT SIN? Y

 742 BYTES FREE

ALTAIR BASIC VERSION 3.1
[FOUR-K VERSION]
OK
```

Empty answers to the first two mean "all of it" and "the default". `WANT SIN?` is asking whether you would like to spend some of your 4K on trigonometry. Say `Y`; you have 742 bytes left and a working `SIN`.

You are in Altair BASIC. It is 1976, and this is the first product Microsoft ever sold.

To do the whole thing in one command, the package ships the machine that does it for you:

```
$ altairsim tapes/basic/basic4k.toml
```

That machine file types those three lines on your behalf. It gets no special powers — every line in it is a line you could have typed.

The sense switches matter

The machine file sets **`sense = 0x80`** on the front panel, and it is not decoration.

The bootstrap reads the sense switches to find out **what device to load from**. Its own printed header says: "*Set A15 on (cassette load), all other switches off.*" A15 on, and nothing else, is `0x80`. Get it wrong and BASIC dutifully loads from **the wrong device**, and then sits there forever waiting for bytes that are never coming, because you told it to listen to a teletype that isn't there.

If a tape load hangs and you are sure the tape is mounted, **check the sense switches first.** The front-panel chapter says where they live.

Speed, and what the crystal actually buys you

The machine runs flat out by default. A cassette that took a real Altair about **110 seconds** to load comes off the tape in **about one**.

You can have the 110 seconds back:

```
altairsim> SET cpu0 clock_hz=2000000
```

That is the real 2 MHz Altair, and the tape now takes as long as a tape took.

Here is the part worth understanding. **What the guest sees is identical either way.** The tape costs the same number of T-states whichever way you run it — the ACR is clocked in T-states, not in wall-clock seconds, and 300 baud means *this many T-states per bit* no matter how fast those T-states are going past. The bytes arrive in the same order with the same gaps between them, measured in the only clock the guest has.

So **the crystal buys you period *feel*, not period *behaviour*.** If you want to watch the tape load at the speed your uncle watched it load, set it. If you want your BASIC prompt, do not. Nothing inside the machine can tell the difference, and no program from the period ever could.

Altair Disk BASIC

> **Not yet in the package.** disks/diskbasic/diskbasic.toml — Altair Disk BASIC, the version that lives on a floppy and has files and `SAVE` — is **TBD**. It is not shipped with this release. Where the rest of this manual mentions it, it is describing something that is coming, not something you have.

The cassette BASIC in the package is the real thing, and it is the one that shows you what an Altair actually was: a box with no operating system, no storage, and no software in it, which you talked into existence one toggled byte at a time.

Serial ports, sockets and telnet

The Altair had no screen. What it had was a serial card, and whatever you chose to hang off it — a Teletype, a glass terminal, a modem, a paper tape reader. The card did not know or care. It moved characters.

`altairsim` keeps that arrangement exactly. **Any board that moves characters has one or more UNITS, and every unit can be CONNECTed to an ENDPOINT.** The unit is the socket on the back of the card. The endpoint is what you plugged into it.

```
altairsim> CONNECT sio0:b socket:2323
altairsim> DISCONNECT sio0:b
```

That is the whole of the interface. The interesting part is the endpoint grammar, and it is short enough to print in full.

The endpoints

This table is exhaustive. There are no others.

Endpoint	Is
`console`	the host terminal — your keyboard and your screen.
`null`	nowhere. Writes vanish. Reads never come.
`loopback`	itself. What the guest writes comes straight back as a read.
`socket:PORT`	**LISTENS** on that TCP port. This is the telnet-in case.
`socket:HOST:PORT`	**CALLS OUT** to that host and that port.
`serial:DEVICE`	a real serial port on this host.

`null` is not an error

An unconnected unit is `null`, and **that is a legitimate state, not a fault.** A 6850 with no cable in it sits there with its transmit register permanently empty, forever ready, and a program that writes to it runs perfectly and talks to nobody. Reads never complete because nothing is sending.

Which is precisely what the real card does with no cable in it. A machine with a second serial port nobody plugged anything into is not a broken machine. `null` models the missing cable, and the guest is entitled to be fooled by it exactly as it would have been in 1977.

The colon is the whole distinction

`socket:2323` **listens.** `socket:localhost:2323` **calls out.** One colon, and it is the same convention every terminal program has used for forty years: a bare port is a port you own, a host and a port is a place you go. Nothing else about the endpoint changes.

Telnetting into the guest

Wire a unit to a listening socket, and the guest has a serial port with a terminal on the end of it. That the terminal is your telnet client, several processes away, is not something the guest can discover.

```
altairsim> CONNECT sio0:b socket:2323
altairsim> RUN
```

Then, from another terminal on your machine:

```
$ telnet localhost 2323
```

The guest is now talking to that window. Your first terminal still has the monitor and `^E` in it. This is how you give a machine two terminals, and it is how you drive a program that wants a console that is not the one you are sitting at.

Calling out

```
altairsim> CONNECT sio0:b socket:bbs.example.com:23
```

The guest dials. As far as the software inside the machine is concerned it has a modem and the modem is connected; it will happily run a period terminal program over it.

A real serial port

```
altairsim> CONNECT sio0:b serial:/dev/tty.usbserial-A600K1XY
altairsim> CONNECT sio0:b serial:COM3
```

The second form is Windows. The bytes go out of a real UART, down a real wire, into whatever you have on the other end.

If you get the device name wrong, it lists the ports that actually exist on your machine. It does not merely say "cannot open". A cable that enumerated under a name one character off from the one you expected is ten minutes of somebody quietly deciding the simulator is broken, and the fix is to print the answer instead of the complaint.

What the card does to the wire

The host port is opened at 9600 8N1, and then **immediately re-programmed by the card** — because the card is the only thing in the system that knows what it is strapped to. The card's `baud`, `data_bits`, `stop_bits` and `parity` become the actual frame on the actual wire.

So if you want 300 baud, 7 bits, even parity going out of that connector, you do not configure it on the connector. You configure it on the **card**, where a 1975 operator would have set it, with a jumper. Modem control lines — DCD, CTS, RTS — are wired through.

And give the machine its real crystal before you transfer a file

```
altairsim> SET cpu0 clock_hz=2000000
```

The moment the wire leaves the machine, the guest is talking to something that keeps time the way *you* do — and the guest does not. It counts instructions. `PCGET` spins a 49-T-state loop to time a second, which is a second at 2 MHz and thirty milliseconds when the machine is running flat out, so it will decide your sender is dead and give up before your sender has drawn breath.

Flat out is the right default for a machine talking to itself. **A machine talking to you wants the crystal.** The troubleshooting chapter has the full story.

A `CONNECT` it does not understand is an error

If `altairsim` cannot make sense of your endpoint, it **refuses, and tells you what it could have meant.** It never quietly falls back to `null`.

This matters more than it sounds like it does. A silent fallback gives you a machine that boots, runs, prints nothing, and hands you a dead terminal to debug — and you will debug the guest, and the card, and the disk, before you think to doubt the thing you typed. A refusal is a worse morning for exactly two seconds. A dead terminal is a worse afternoon.

Exactly one unit may hold the console

The console is **your keyboard**, and there is one of it.

```
altairsim> CONNECT sio1:a console
console: taken from sio0:a
```

Connecting a second unit to `console` **steals it, and says who it took it from.** It is not an error and it is not shared. Two boards reading one keyboard would each get roughly half the characters, in an order neither of them could predict, and the resulting machine would appear to be haunted.

To see who has it:

```
altairsim> SHOW CONSOLE
```

That also shows the transforms, which is the rest of this chapter.

The transform chain belongs to the console, and only the console

This is the most important rule in the chapter, and it is worth stating twice before explaining it.

The `[console]` settings are the only thing in the simulator that alters a byte. Every serial LINE is 8-bit clean. There is no knob anywhere on any card that masks a bit.

Setting	Does
`upper`	folds what you type to upper case
`strip7in`	clears bit 7 of every character you type
`strip7out`	clears bit 7 of every character the guest prints
`crlf`	translates line endings
`echo`	echoes your keystrokes locally
`bell`	rings the terminal bell on `^G`
`bsdel`	swaps backspace and delete
`attn`	which control character is ATTN (default `^E`)

Set them with `CONSOLE k=v`. (`SET CONSOLE k=v` is the same thing said longer.)

```
altairsim> CONSOLE strip7out=on
altairsim> CONSOLE upper=on crlf=off
altairsim> CONSOLE attn=1D
```

`attn=1D` moves the escape key from `^E` to `^]`. **It must be a control character** — an ATTN you can type by accident in the middle of a sentence is not an escape key, it is a trap.

Why it works this way: `MEMORY SIZ?`

Boot MITS BASIC with the transforms off and it asks you:

```
MEMORY SIZ?
```

The `E` is missing, and there is a garbage character where it should be. BASIC is not broken. **MITS BASIC sets bit 7 of the last character of every message**, as a string terminator — that is how it knows where a string ends — and it sends it. `E` is `45`; with bit 7 set it is `C5`, and your terminal prints whatever `C5` happens to mean to it.

The fix is `CONSOLE strip7out=on`, and the reason the fix lives *there* is the whole argument:

On the real machine, the card sent all eight bits. Nothing masked anything. The card put `C5` on the wire because that is the byte BASIC handed it. The Teletype on the other end simply **did not look at bit 7** — on a Model 33, that is the parity position, and the printing mechanism does not decode it. It printed `E` and threw the eighth bit on the floor.

Nothing was masked. Something on the far end did not look. **`strip7out` is the terminal not looking.** It is a property of the thing you are sitting at, and it belongs on the thing you are sitting at.

Why not just strap the card to 7 bits

Because it would work, and then it would silently destroy your data.

Set a 7-bit mask on the card — or a filter on the line — and BASIC's prompt comes out clean. It also **silently corrupts every XMODEM transfer through that port**, because XMODEM sends binary, every byte of it matters, and bit 7 is a real bit in half of them. The file arrives. The checksums even pass on a bad packet often enough to be maddening. And the corruption is in the *plumbing*, which is the last place anyone looks.

A line may carry binary. A terminal is not a line. The transform belongs to the terminal because only the terminal knows it is displaying text.

`data_bits` and `parity` are real hardware, and are not this

A card genuinely does have `data_bits`, `stop_bits` and `parity`, and they are genuinely configurable, because a 6850 genuinely has those straps. They are **a FRAME** — they describe what physically travels down the wire, bit by bit, and on a real serial port they are what the far end must agree to or it will read garbage.

They are never a mask. `data_bits=7` is not "and the byte with `7F`". It is "put seven data bits in the frame", which is a statement about the wire, not about the byte the guest wrote. Do not reach for it to fix a prompt.

Moving files in and out

You will want to get a file into CP/M, and you will want to get one back out. There is a card for it.

The Host Bridge is ours, and it is not a period card

Say it plainly: **MITS never made this.** No S-100 vendor made this. The `hostbridge` card is **our own**, invented for this simulator, and it exists because the alternative — a modem protocol over a simulated serial port at 300 baud — is a slow, fiddly answer to a question that only exists because the machine is simulated in the first place.

Everything else in this program is a board somebody could buy in 1977. This one is not, and you should know which is which. It sits in the manual next to the real cards, and it is the only one wearing a different badge.

It is in the **`default` machine**, so unless you have written a machine file that leaves it out, **you already have it.**

Default port **B0**: `BASE+0` is command/status, `BASE+1` is data.

The three utilities, and where they live

The utilities are **on the disk**, not in the card. They are ordinary CP/M `.COM` files, and they were assembled inside the machine, by the machine's own assembler, from source that ships with it.

`HDIR [pattern]`	List what is on the host.
`R <hostfile> [cpmfile]`	Read host → CP/M.
`W <cpmfile> [hostfile] [B|T]`	Write CP/M → host.

`HDIR`

```
A>HDIR
A>HDIR *.ASM
```

`HDIR` always prints the TRUE host names — the real ones, with their real case and their real length, not the 8.3 names CP/M is going to see them as. That is the point of it. When you cannot work out what to call a file, ask `HDIR` what it is actually called.

`R` — host to CP/M

```
A>R README.TXT
A>R *.ASM
A>R SRC/FOO.ASM
A>R SRC/FOO.ASM WORK.ASM
```

Wildcards work. Subdirectories work — and **both `/` and `\` are accepted on every host**, because you should not have to remember which machine you are sitting at. A second argument names the file on the CP/M side and overrides the automatic mapping.

`W` — CP/M to host

```
A>W GAME.COM
A>W GAME.COM game.com
A>W NOTES.TXT notes.txt T
```

`W` defaults to B, and that is the important sentence

Binary is the default. `W` writes **every byte of every record, exactly** — nothing is inspected, nothing is stripped, nothing is guessed. A `.COM` file survives the trip and runs.

The cost is visible and it is honest: **a text file arrives with up to 127 trailing `^Z` bytes** on the end, because CP/M stores files in 128-byte records and pads the last one, and `W` in binary mode does not presume to know which of those bytes you meant. Your editor will show them. Trim them, or ask for `T`.

`T` (text) stops at the first `^Z`. You get clean text, exactly as long as it should be.

And now the trap, which is the reason `T` is not the default: **a binary file containing byte `1Ah` comes back TRUNCATED.** Byte `1Ah` is `^Z`. It is a perfectly ordinary byte in a `.COM` file — it is `LDAX D` — and in text mode the transfer stops dead at the first one, silently, and hands you a file that is the right name and the wrong length.

Some emulators default to text and keep a list of extensions to exempt. **That silently truncates files** the day you transfer something whose extension is not on the list, and you find out weeks later. We would rather hand you a text file with some padding on it than hand you half a program and call it done.

Mode	What it does	Costs you
`B` (default)	Every byte, exactly.	Up to 127 `^Z` bytes on a text file.
`T`	Stops at the first `^Z`.	**Truncates any binary containing `1Ah`.**

The sandbox

The card has a **`hostdir`** property. It is the host directory the guest can see, and it is the **only** host directory the guest can see.

```
altairsim> SET hb0 hostdir=/tmp/xfer
```

Empty — which is the default — means **the directory you ran `altairsim` from.**

Where a relative `hostdir` points

`hostdir` is a path, so it obeys the path rule from *Machines* — **both halves of it**, and this is the one place the difference has teeth, because this path is a fence and the other end of it is a CP/M program.

```
altairsim> SET hb0 hostdir=xfer          # you typed it -> ./xfer, from YOUR shell
```

```
[[board]]
id      = "hb0"
hostdir = "xfer"                          # the file wrote it -> the xfer beside THIS FILE
```

Both say `xfer`. They are **different directories**, and each is the one its author could see — which is the same rule that decides where `mount = "cpm.dsk"` looks. The machine file's `xfer` travels with the machine file, so an example directory you copy to your desktop still has its own transfer folder.

You never have to work it out. The written value and the resolved one are both printed, and the resolved one is the fence:

```
altairsim> SHOW hb0
  property         value            legal
  port             0xB0             0..254
  hostdir          xfer
  hostdir_root     /home/you/altair/disks/cpm22/xfer (read-only)
  readonly         false            true|false
```

`hostdir` is what was **written**. `hostdir_root` is where the fence **actually is** — read-only, because it is a fact about this run rather than a setting, and it is never written back into a machine file.

`SHOW PATHS` prints the same root beside the other two bases. If `R` cannot find a file you are certain you put in `xfer`, that is the line to read: there is very likely a second `xfer`, and the guest is standing in the other one.

…and what that means for `R` and `W`

The path rule stops at the card. **It does not reach the guest, and `R` and `W` never see it.**

At the `A>` prompt you are not typing a host path at all — you are typing a **name**, and the card resolves it inside `hostdir_root` and nowhere else:

```
A>R FOO.ASM          <- hostdir_root/FOO.ASM. Never your cwd, never the machine file's
A>W RESULT.TXT       <- hostdir_root/RESULT.TXT
```

So the path rule reaches `R` and `W` **exactly once**: it decides which directory `hostdir` named, back when someone wrote it. After that the guest has one directory and no way to say anything about any other — `..`, an absolute path and a drive letter are all refused at the card, as below.

This is worth stating plainly because the two mechanisms look alike and are not:

	decides	**confines**
the path rule	where a path *written by a human* points	nothing at all
`hostdir`	nothing you type	**everything the guest can reach**

This is the *only* fence in the simulator, and it is worth not confusing with the path rule in *Machines*. That rule decides where a path written in a machine file points, and it confines nothing at all. This one decides how far the **guest** can reach, and confines everything. A machine file may mount a disk from anywhere on your system; the CP/M program running off that disk still sees only `hostdir`.

The guest cannot escape it. All of the following are refused, at the card, before anything touches your filesystem:

- an absolute path
- a drive letter
- any `..` component
- a symlink that resolves outside the root

This is a **deliberate, hard boundary**, not a best-effort filter. The guest is running software from 1977 that you found on the internet, and the answer to "what if it is hostile" should not be "well, it probably isn't."

`readonly` makes it a one-way street:

```
altairsim> SET hb0 readonly=on
```

Files come **out of** the host. Nothing goes back in. `W` fails.

And it means the guest can write your working directory

Here is the other half of that, and it does not get buried:

By default, guest software can read and write the directory you ran `altairsim` from.

That is a real capability and it is on unless you turn it off. It is a **deliberate trade**, not an oversight: `R FOO.ASM` working the instant you unzip the package, with no setup, is worth a great deal, and the directory you are standing in is the directory you almost always meant. But it is your directory, with your files in it, and a CP/M program you did not write can reach them.

If that is not what you want, point `hostdir` somewhere else, or set `readonly=on`. Both are one line.

Names: 8.3, and how a host name becomes one

CP/M has eight characters and an extension of three, and your host does not. So the card maps:

1. **uppercase** the host name,
2. truncate the base to **8**,
3. truncate the extension to **3**,
4. **drop illegal characters.**

```
my-notes(2).txt   →   MYNOTES2.TXT
```

A **second argument overrides the whole business**, and when the mapping produces something you do not like, that is what it is for:

```
A>R my-notes(2).txt NOTES.TXT
```

And again: `HDIR` prints the **true** host names, so you can always find out what you are actually asking for.

Case, and why the card does not guess

The CP/M command line folds everything to upper case. This is not something we do; it is what the CCP does, before your program ever sees the line. By the time `R` runs, `R readme.txt` and `R README.TXT` are **the same command**. The information is gone.

So the card compensates, in this order:

1. **Exact match wins.** If there is a file called exactly what you asked for, that is the file.
2. **Otherwise, fold case and look again.** `README.TXT` will find `readme.txt`.
3. **If several files match, it REFUSES.**

Step 3 is the one that matters. If your directory holds `readme.txt` and `README.TXT` and `ReadMe.Txt`, the card does not pick one, does not pick the newest, and does not pick the first. **It tells you it cannot tell**, and stops. A file transfer that guesses wrong and tells you it succeeded is worse than one that fails.

Why one `R.COM` works on every disk you will ever build

Every file operation goes through CP/M's BDOS. Not the BIOS. Not a disk parameter block. Not an assumption about how many sectors are on a track or where the directory lives.

`R` and `W` open, read, write and close through the same BDOS calls any CP/M program uses, and the BDOS is the thing that knows about your disk. Which means the same `R.COM`, unmodified, runs on:

- the 8″ floppy controller,
- the 8 MB disk,
- the minidisk,
- and **any BIOS anybody writes later**, for a controller that does not exist yet.

That was the design goal. The card is not a period card, but the software that drives it is a perfectly well-behaved CP/M program, and it will still work on the machine you have not built yet.

Worked examples

Two complete sessions. **Every transcript below was captured from the program**, not typed out from memory — if it says the machine printed something, the machine printed it.

---

1. CP/M from a floppy

```
$ altairsim disks/cpm22/cpm22-buffered.toml
```

```
startup> RUN FF00
[console -- ^E returns to the monitor]

56K CP/M 2.2b v2.3
For Altair 8" Floppy

A>
```

What is on the disk

```
A>DIR
A: L80      COM : LADDER   COM : ED       COM : ASM      COM
A: DUMP     COM : XSUB     COM : PCGET    COM : LS       COM
A: SUBMIT   COM : LOAD     COM : SURVEY   COM : VIEW     COM
A: LADDER   DAT : LUNAR    BAS : M80      COM : MAC      COM
A: MBASIC   COM : PIP      COM : STAT     COM : DDT      COM
A: MOVCPM8  COM : NSWP     COM : SYSGEN   COM : ACOPY    COM
A: OTHELLO  COM : STARTRK  BAS : TICTAK   BAS : WM       COM
A: WM       HLP : CRC      COM : PCPUT    COM : AFORMAT  COM
A: STARINS  BAS : IOBYTE   TXT
A>
```

A macro assembler, a linker, Microsoft BASIC, `DDT`, `PIP`, a text editor, and Star Trek. It is a working 1977 development system.

Out, and back in

`^E` stops the machine and gives you the monitor. Nothing is lost — a bare `RUN` resumes at the instruction it was about to execute.

```
A>
ATTN -- the machine is still at CA9C. RUN resumes.
C0Z1M0E1I0 A=00 B=007F D=CA01 H=BC0E S=BC37 IE=1 P=CA9C  CALL CA78
altairsim>
```

Look at the machine while it sits there:

```
altairsim> BOARDS
altairsim> SHOW dsk0
altairsim> DUMP 100
```

…and hand the keyboard back:

```
altairsim> RUN
```

Your `A>` prompt is exactly where you left it.

Before you write anything

The disk is mounted **read/write** and there is no undo. Copy the folder first — it is self-contained and boots from anywhere:

```
$ cp -R disks/cpm22 my-cpm
$ altairsim my-cpm/cpm22-buffered.toml
```

And when you are done writing files, **get back to the `A>` prompt before you quit.** The BIOS holds a track in memory and only flushes it when it next reads the console. The disks chapter explains why.

---

2. Altair BASIC from a cassette

```
$ altairsim tapes/basic/basic4k.toml
```

```
startup> MOUNT acr0:tape "4K BASIC Ver 3-1.tap"
acr0:tape: mounted 4K BASIC Ver 3-1.tap
startup> LOAD "LDR4K31.HEX"
loaded 20 bytes from LDR4K31.HEX (0000-0013)
startup> RUN 0
[console -- ^E returns to the monitor]

MEMORY SIZE?
TERMINAL WIDTH?
WANT SIN? Y

742 BYTES FREE

ALTAIR BASIC VERSION 3.1
[FOUR-K VERSION]

OK
PRINT 6*7
 42

OK
```

Seven hundred and forty-two bytes free. That is the machine Microsoft was founded on.

What those three startup lines actually are

They are not magic, and they are not a boot command — **there is no boot command.** They are the three things an operator did in 1975, written down:

`MOUNT acr0:tape "…"`	**Put the cassette in the recorder and press PLAY.**
`LOAD "LDR4K31.HEX"`	**Toggle in the bootstrap.** Twenty bytes, entered by hand on the front-panel switches. MITS printed them in the manual.
`RUN 0`	**Set the switches to zero, press EXAMINE, then press RUN.** EXAMINE is what puts the switches into the program counter; without it RUN carries on from wherever the machine already was.

Anything you can type at the prompt, a machine file can do. It gets no special powers, and that is why you can do this yourself:

```
altairsim> REWIND acr0:tape
altairsim> RUN 0
```

`REWIND` is a verb the **cassette card brings with it** — it exists only because there is an ACR in the machine. You need it to load the same tape twice, for exactly the reason you would have

needed it in 1975.

The tape is not in the machine file

Look again at what the machine file declares: a front panel, a processor, a serial card, a cassette interface, and some memory. **It does not declare the tape.**

A machine file describes **hardware**. Which cassette is in the recorder is not hardware — and there is no motor control on the card to make it one. You put the tape in, and you press PLAY. That is what `MOUNT` is.

One number in that file you could not have guessed

The front panel's `sense` switches are set to `80`. That is `A15` up, and nothing else.

The bootstrap's own printed header says why:

```
** Set A15 on (cassette load) **
** All other switches off **
```

`A15` up means *load from the cassette*. Get it wrong and BASIC comes up talking to the wrong device, or to nothing at all. Period software reads those switches, so they are not decoration.

It loaded in a second, and a real one took two minutes

The machine runs **flat out** by default. A 300-baud cassette that took a real Altair 110 seconds comes off in about one.

If you want the real thing:

```
altairsim> SET cpu0 clock_hz=2000000
altairsim> REWIND acr0:tape
altairsim> RUN 0
```

…and now it takes 110 seconds, because the tape costs the same number of T-states either way. **What the guest sees is identical.** The crystal buys period *feel*, not period *behaviour*.

Where to go next

- **Move a file between CP/M and your own machine** — the file-transfer chapter (`R`, `W`, `HDIR`).

- **Telnet into the guest, or wire it to a real serial port** — the serial chapter.
- **Look at the bus while it runs** — the debugging chapter.

> **Altair Disk BASIC** is not in the package yet. When it is, it will boot the same way everything else here does: name its machine file, and it runs.

The MCP server

```
$ altairsim --mcp
```

That runs `altairsim` as an **MCP (Model Context Protocol) server** on stdin and stdout, so that an AI assistant — Claude, or anything else that speaks MCP — can drive the machine through **typed, structured tools** instead of screen-scraping a text terminal.

It is not a wrapper, and it is not a second model of the world. **The MCP server runs on the same machine object as the monitor.** What the tools see is exactly what `SHOW` sees, because it is the same machine, answering the same questions, through a different door.

What the tools do

Enough to operate the machine:

- List the board types available, and every property each one has — **with its type, its default and its legal range.**
- List the boards actually in the machine.
- Get and set any property on any board.
- Add a board.
- Examine and deposit memory.
- Run the machine, and step it.

The five you will see named are `board_types`, `board_list`, `board_get`, `board_set` and `board_add`. The rest follow the monitor's own vocabulary.

Driving a running guest

Building a machine is half of it; the other half is **operating one that is running** — typing at its console and reading what it prints. Four tools do that, and they are what let an assistant boot CP/M, run `ASM`, and talk to a program over a serial port entirely through MCP:

- **`run`** — advance the guest a bounded slice and return what it printed. It is the expect loop in one call: pass `input` to type a line, `until` to stop when a string (a prompt like `A0>`) appears, `from` to set the PC first (booting is `from` the boot PROM). It **also stops on its own when the guest reaches a prompt** — spinning on the console with nothing to say — so you get control back without guessing a timeout. Every stop says why in `stopped`: `match`, `idle`, `timeout`, `steps`, `halt`, `breakpoint`.
- **`send`** — type at the console without running (then `run` to let it be read).

- `recv` — drain what the guest has printed since you last looked, without running.
- `regs` — the CPU registers right now.

The shape of a session is therefore: `run {from: 0xFF00, until: "A0>"}` to boot, then `run {input: "ASM FOO\r", until: "A0>"}` per command, reading the reply each time. A `run` **never blocks** — it runs the guest flat out for at most `timeout_ms` (default 2000) and returns — so a `tools/call` always comes back, unlike a bare `RUN` through the `monitor` tool, which under a pipe waits on a stdin that is the JSON-RPC channel itself.

Under `--mcp` the console line is quietly re-seated onto an in-memory terminal the server owns (there is no host keyboard behind a pipe), which is what `send` / `run` / `recv` read and write. Everything else on the machine — a second serial card wired to a real port, a socket — keeps running and is serviced on every `run` slice, so a program shuttling bytes between the console and a modem port works exactly as it would at a real terminal.

The schemas describe themselves

Every tool's schema comes off the same reflection layer as the TOML keys and the `SET` / `SHOW` commands. There is one description of what a board is and what it can be asked, and the machine file parser, the monitor, and the MCP server all read it.

The consequence is the point: **a board added tomorrow is drivable by an assistant the day it lands, with no new code.** Nobody writes an MCP tool for the new card. The card declares its properties, as it must anyway to be configurable at all, and the tool schema is that declaration.

So there is no tool reference in this manual. Start the server and ask it what it has — `tools/list` returns every tool the server exposes, and `board_types` every board type; their answers are authoritative in a way a printed list could never be.

Configuring an assistant to use it

MCP clients differ, but they all want the same two things: a command to run, and the fact that it speaks over stdio. The command is `altairsim --mcp`, and it does.

Troubleshooting

Most of what goes wrong here is not a fault. It is the machine doing exactly what a real one did, in a way nobody warned you about. This chapter is the warning, after the fact.

`^C` does not stop the machine

It is not supposed to.

`^C` belongs to the guest. CP/M reads it — it is how you warm-boot, and how you break out of a BASIC program. A stop key the guest also wants is a stop key the guest will eat, and then you are trapped inside your own simulator.

`^E` is the stop key. It is ATTN, the host intercepts it before the guest is ever offered the byte, and **no program running inside the machine can disable it, ignore it, or take it from you.**

If a guest genuinely needs `^E` for something of its own, move ATTN out of its way:

```
altairsim> CONSOLE attn=1D
```

That makes it `^]`. It must be a control character.

I pressed ATTN and now the machine seems stuck

It is not stuck. It is **stopped**, which is what ATTN is for, and nothing has been lost.

ATTN halts the processor and hands you the monitor. It is not RESET and not POWER: the registers, the memory and the disk are precisely as the guest left them, and the monitor told you where to pick it up when it printed "*still at* ...".

```
altairsim> RUN
```

A bare `RUN` — no address — resumes at that exact instruction, and your `A>` is where you left it. (`RUN <addr>` is a different thing: it loads the PC first, so it *restarts* rather than resumes.)

And while you are stopped, you can look: `REGS`, `EXAMINE`, `DUMP`, `DISASM` and `STEP` all work at this prompt, on a machine that is not moving under you. See the debugging chapter. `BREAK` is for stopping at a place you cannot reach by hand.

My file was not written / the disk lost my changes

You quit too early.

The CP/M in this package has a **track-buffering BIOS**: it keeps a whole track in memory and only flushes it to the image when it next reads the console. This is not a shortcut in the simulator; it is what that BIOS does, and it is why the machine was fast. (Not every CP/M is built this way — some write each sector as it goes — but you cannot tell from the prompt which one you are sitting at, so assume you are on a buffered one.)

Get back to the `A>` prompt before you quit or copy the image. The moment CP/M asks you for a keystroke, the buffer has landed. Kill the program the instant a file operation appears to have finished and the last track never reached the disk. The disks chapter has the detail.

The disk image changed and I wanted it not to

There is no undo. **The image is mounted read/write, like a real machine** — a CP/M that cannot write its disk cannot save your work.

Two answers, and take one *before* you experiment:

```
altairsim> MOUNT dsk0:drive0 file.dsk RO
```

`RO` refuses every write at the controller, so your file is safe whatever the guest does. It is for a disk you mean to **read**: the guest is never told the disk is protected — the controller has no bit that says so and CP/M has no error that means it — so a program that sets out to write to an `RO` disk will not fail gracefully. The disks chapter explains why.

or copy the folder. It is a directory with a machine file and an image in it, and it boots from anywhere:

```
$ cp -R disks/cpm22 my-cpm
```

`MOUNT` says "no such file" and the file is right there

Look at *where you typed it from*.

A path you TYPE is relative to YOUR SHELL. A path inside a machine file is relative to THAT MACHINE FILE. Those are two different directories, and one of them is not the one you are thinking about.

This is deliberate, and it is the reason an example folder boots from anywhere: the machine file says `cpm22.dsk` and means *the image next to me*, so you can move the folder, or run it from three levels up, and it still finds its disk. If a typed path resolved the same way, you could not mount a file from your own working directory without knowing where the machine file happened to live.

Every prompt ends in a garbage character

```
MEMORY SIZ?
```

That is **MITS BASIC setting bit 7 of the last character of every message** as a string terminator, and your terminal printing the result. The `E` is there; it arrived as `C5` instead of `45`.

```
altairsim> CONSOLE strip7out=on
```

The real Teletype ignored bit 7 and printed the `E`. `strip7out` is your terminal doing the same. The serial chapter explains why the fix belongs on the console and **never** on the card.

Nothing appears when I type / the guest does not see my keys

Ask who has the keyboard.

```
altairsim> SHOW CONSOLE
```

Exactly one unit may hold the console. If the console is on a unit the guest is not reading, you are typing into a card nobody is listening to. Connect the right one — and note that connecting a second unit to `console` *steals* it from the first, and says so.

The machine runs impossibly fast — a cassette loads in one second

It is running **flat out**, which is the default. `clock_hz = 0` means "no divisor, go".

```
altairsim> SET cpu0 clock_hz=2000000
```

That is the real 2 MHz Altair, and it will give you the real 110-second cassette load. Both behaviours are correct; one of them is just a much longer lunch.

A file transfer keeps timing out — `PCGET`, `PCPUT`, XMODEM

Slow the machine down. Transfers with the outside world want the real crystal.

```
altairsim> SET cpu0 clock_hz=2000000
```

Here is why, because it is worth understanding once and it explains a whole class of symptom.

A guest program has no clock. It counts instructions. `PCGET` times a second the way every program of the period timed a second — by spinning in a loop and counting the trips. Its own source says so:

```
MSEC    lxi  d,(159 shl 8)   ;49 cycle loop, 6.272ms/wrap * 159 = 1 second
```

That arithmetic is *only* a second if the crystal is 2 MHz. `PCGET` waits three of them for a block header. Run the machine flat out and those three seconds — three seconds of **T-states** — are retired by your host in a few tens of **milliseconds**, while the sending program on the other end of the wire is still living in seconds of the ordinary kind. `PCGET` concludes the sender is dead, NAKs, purges the line, and gives up. Nothing is broken. The two ends are simply no longer using the same clock.

And that is the boundary, exactly: timing inside the machine is consistent at any speed, because everything in there counts the same T-states — which is why a cassette loads correctly flat out, the ACR and the tape agreeing with each other at whatever speed the pair of them run. A transfer to your host has **one end inside the machine and one end outside it**, and only the crystal makes those two agree.

So: flat out for everything the machine does to itself. The real 2 MHz for anything it does with you. Set it back afterwards if you like the speed — and note the built-in file-transfer card is not affected by any of this, having no timeouts to expire.

Two cards are fighting over a port

```
altairsim> SHOW BUS CONTENTION
```

names them. To see the whole map of who decodes what:

```
altairsim> SHOW BUS IO
```

On a real S-100 machine this was two cards strapped to the same address and a bus you could not trust. Here it is a list.

An `IN` from a port returns `FF` and I expected something

Nothing decodes that port. The bus floats high when no card is driving it, and `FF` is what a floating bus reads. It is not an error, and the machine will not tell you — a real one did not either.

To find out who *would* have answered, without running a cycle:

```
altairsim> WHO IO 10
```

`RESET` did not clear memory

It is not supposed to.

`RESET` is the bus's RESET* line. It does what pulling that line did: the processor goes to zero, cards return to their power-on state. RAM is RAM; it was not cleared on the real machine and it is not cleared here.

`POWER` is the only thing that loses memory. That is the difference between pressing a button and pulling a plug, and the manual keeps it.

macOS refuses to run it

"*cannot be opened because the developer cannot be verified*".

```
$ xattr -dr com.apple.quarantine ./altairsim
```

Once. It is not a comment on the program; it is what macOS does to every unsigned binary that arrives in a zip.

If that prints nothing, it worked — and if the flag was never there (a zip fetched with `curl` or `scp` is not marked; only one a browser or a mail client downloaded is), it also prints nothing and succeeds. That is the whole reason for `-dr` over a plain `-d`, which announces an error when there is nothing to remove.

Glossary

ACIA — Asynchronous Communications Interface Adapter. The Motorola 6850 chip at the heart of the 88-2SIO serial card. It has two registers a program can see: a status register and a data register. Everything a program does with a serial port on this machine, it does through one of those.

ACR — Audio Cassette Recorder interface. The MITS 88-ACR: a card that wrote bits to an ordinary audio cassette and read them back. It is how you loaded BASIC if you could not afford a floppy, which in 1976 was most people.

ATTN — Attention. The key that stops a running guest and returns you to the monitor. It stops the machine but does not disturb it — not RESET, not POWER — so a bare `RUN` resumes at the instruction it was about to execute. It is `^E` by default, the host intercepts it before the guest sees the byte, and no guest program can take it from you. Move it with `CONSOLE attn=`.

backplane — The board with the connectors on it that every card plugs into. On an Altair it is the S-100 bus itself: eighteen slots, one hundred pins each, all wired in parallel. There is no chipset. The backplane *is* the machine.

bank switching — Making more memory than the processor can address by putting several banks at the same addresses and switching between them. An 8080 can address 64K and no more. A machine with 128K of RAM in it lies to the processor about which half it is looking at.

BDOS — Basic Disk Operating System. The middle layer of CP/M: files, directories, the console. A CP/M program asks the BDOS for things and does not know what hardware answers.

BIOS — Basic Input/Output System. The bottom layer of CP/M, and the only part that knows what machine it is on. Move CP/M to a new computer and the BIOS is what you rewrite; the BDOS and the CCP are untouched. It is also where a track buffer lives *if the author put one there* — which is why, on some CP/Ms and not others, a file can be "written" and not yet be on the disk. See the disks chapter.

CCP — Console Command Processor. The top layer of CP/M — the part that prints `A>` and runs what you type. It is deliberately expendable: a big program is allowed to overwrite it, and CP/M reloads it from disk afterwards. That reload is the warm boot.

contention — Two cards answering the same address. On a real S-100 machine both would drive the data bus at once and you would get a byte that is neither of theirs, intermittently, in a way that would take you a week. `SHOW BUS CONTENTION` names them instead.

CP/M — Control Program for Microcomputers. Digital Research's disk operating system, and the reason the 8080 mattered. Write a program for CP/M and it ran on every 8080 machine ever

built, whoever built it — the first time that was true of anything.

DBL — Disk Boot Loader. The boot PROM on the MITS floppy controller, at `FF00`. It reads one sector off track 0 and jumps into it. **There is no `BOOT` command on an Altair** — you set the address switches to `FF00`, press EXAMINE to load them into the program counter, then press RUN, and this is the thing you are running.

decode — What a card does when it recognises an address as its own and answers. A card that does not decode an address stays silent and lets somebody else have it. Which card decodes which address is the entire question of how an S-100 machine is put together.

DMA — Direct Memory Access. A card taking the bus away from the processor and reading or writing memory itself, without the processor's help. Faster, and the origin of some spectacular bugs.

endpoint — In this program, the thing on the far end of a serial unit's cable: `console`, `null`, `loopback`, a TCP socket, or a real serial port. The serial chapter has the complete list; there are no others.

floating bus — What the data bus reads when nothing is driving it. On the S-100 it floats high, so an `IN` from a port no card decodes returns `FF`. That is not an error. That is the absence of a card, and it looks like `FF`.

front panel — The switches and lamps on the front of the Altair. The address switches, the data lamps, DEPOSIT, EXAMINE, RUN, STOP, RESET. Before there was a terminal, this *was* the user interface, and you toggled your bootstrap in through it one byte at a time. In this program it is a card like any other.

FSK — Frequency Shift Keying. Encoding bits as two audible tones — one for a zero, one for a one. It is how the ACR got data onto a cassette, and it is why a loading tape sounds the way it does.

hard-sector — A floppy where the sector boundaries are marked by physical holes punched in the disk, and the controller counts them going past. The MITS floppies are hard-sectored. A soft-sectored disk has one hole and finds its sectors by reading marks written in the data, which is the arrangement that won.

IntAck — Interrupt Acknowledge. The bus cycle the 8080 runs when it accepts an interrupt. The interrupting card puts one instruction on the data bus during that cycle, and the processor executes it. Almost always an `RST`.

MCP — Model Context Protocol. The protocol an AI assistant uses to call structured tools. `altairsim --mcp` speaks it, so an assistant can drive the machine directly. See the MCP chapter.

monitor — The `altairsim>` prompt. Not a program running inside the machine — it is you, standing in front of it, with a much better front panel than MITS shipped.

PHANTOM* — An S-100 line that tells memory cards to shut up. Pull it, and a ROM can answer at an address a RAM card also decodes, without contention. It is how a boot PROM can sit on top of RAM and then get out of the way. The asterisk means the line is active low, and on this bus most of them are.

pINT — The S-100 interrupt request line. Any card may pull it. The processor notices, if interrupts are enabled, and runs an IntAck cycle to find out who and why.

PROM — Programmable Read-Only Memory. A chip with a program burned into it that survives power-off. The Altair's boot loader lives in one, which is the only reason a disk machine can start at all: something has to already be there to read the disk.

RST — Restart. A one-byte 8080 instruction that calls a fixed low address — `RST 0` through `RST 7`, at `0000`, `0008`, and so on to `0038`. One byte, so it fits in an IntAck cycle, which is exactly what it is for.

S-100 — The bus. One hundred pins, eighteen slots, designed by MITS for the Altair and then adopted by everyone. It was the first open standard in personal computing, mostly by accident, and it is the reason a card from one company worked in another company's machine.

sector — The smallest chunk of a disk you can read or write. On these floppies, 137 bytes of which 128 are yours.

SENSE switches — The top eight address switches (A8–A15) on the front panel, readable by a program as an input port. Software used them for configuration before there was anywhere else to put it: which port is the console, how much memory to use, whether to load from tape. Half the boot procedures in this manual begin with a sense switch setting.

T-state — One clock cycle. Every 8080 instruction costs a known number of them, and that is how this program knows what time it is. At 2 MHz a T-state is 500 nanoseconds, and a cassette takes 110 seconds because it takes 220 million of them.

TDRE — Transmit Data Register Empty. The bit in the 6850's status register that means "the card has room for another character". A program that wants to print polls it until it sets. A serial port connected to `null` sets it forever, which is why writing to nothing works fine.

track — One concentric ring of sectors on a disk. The head steps in and out to reach a track and does not move again to reach the sectors on it, which is why a BIOS that buffers buffers a whole track at a time: having paid for the seek, it may as well have the lot.

UART — Universal Asynchronous Receiver/Transmitter. The chip that turns a byte into a sequence of bits on a wire and back again. The ACIA is one.

unit — One channel on a card that moves characters — the socket on the back. A 2SIO has two of them, `a` and `b`, and they are connected independently. `CONNECT sio0:b` names the board and then the unit.

VI0–VI7 — The eight vectored interrupt lines on the S-100 bus, served by the 88-VI/RTC card. **VI0 is the highest priority.** A card pulling VI*n* gets `RST` *n*, and the interrupt card sorts out who wins when two pull at once.

warm boot — CP/M reloading its own top layer (the CCP) from the disk, because the program that just finished was allowed to overwrite it. `^C` at the `A>` prompt does one deliberately. It is not a reset; the machine never stopped.

Every monitor command

Commands resolve by prefix, and the first match wins. There are no aliases and no fixed abbreviations: the shortest prefix that reaches a command is derived from the table's order, so it is shown here as `D[UMP]` — type the part before the bracket. (`?` is the one true alias, for `HELP`.)

Numbers: on the wire is **hex** (addresses, ports, bytes); never on the wire is **decimal** (counts, widths, sizes). `0x` / `$` / `h` force hex, `#` forces decimal, and a `K` / `M` suffix is always decimal.

Not built yet

These **resolve** — they take their abbreviation today, so it cannot change under your fingers when they land — and they tell you what they are waiting on.

Command	Waiting on
`E[DIT]`	the line editor
`STO[P]`	a monitor that runs alongside the machine (ATTN leaves CONSOLE today)
`SN[APSHOT]`	the debugger
`REST[ORE]`	the debugger
`REC[ORD]`	the debugger
`REP[LAY]`	the debugger

The commands

DUMP — `D[UMP]`

```
DUMP [<addr>|<range>] [WIDTH=16]
```

Hex and ASCII. A bare address runs to the END OF ITS PAGE, and a bare DUMP continues from there -- so the rows and the columns both stay page-aligned however you first landed. WIDTH is a count, so it is decimal.

```
D 100        0100-01FF, a whole page
D 0001       0001-00FF: stops on the boundary, last line full
D            the next page
D FF00-FF0F  an explicit range means exactly what it says
D 100/20     0100-011F (LEN is part of the address expression: hex)
D 0 WIDTH=8  eight bytes per line
```

STEP — `S[TEP]`

```
STEP [n]
```

One instruction, with REAL bus cycles through the real decode. Prints each instruction as it goes; past 32 it runs quietly and reports. `n` is a count, so it is decimal.

A LINE IS THE STATE BEFORE THE INSTRUCTION ON IT RUNS -- the registers as they stand, and then what is about to happen to them. So a value you just loaded appears on the NEXT line, and the line the monitor leaves you on is where you have stopped: that instruction has not run.

```
S            one instruction
S 10         ten of them
```

```
altairsim> DEPOSIT 0 3E 05 06 0A 80 76        MVI A,5 / MVI B,0A / ADD B / HLT
altairsim> S 3
C0Z0M0E0I0 A=00 B=0000 D=0000 H=0000 S=0000 IE=0 P=0000  MVI A,05
C0Z0M0E0I0 A=05 B=0000 D=0000 H=0000 S=0000 IE=0 P=0002  MVI B,0A
C0Z0M0E0I0 A=05 B=0A00 D=0000 H=0000 S=0000 IE=0 P=0004  ADD B
C0Z0M0E1I0 A=0F B=0A00 D=0000 H=0000 S=0000 IE=0 P=0005  HLT
```

Three instructions ran; the fourth line is the HLT waiting. A=05 shows up one line below the MVI that loaded it. The flags are the 8080's own five, in the Altair's lettering -- Carry, Zero, Minus, Even parity, Interdigit carry -- and E goes to 1 on the ADD because 0F has an even number of bits set.

NEXT — `N[EXT]`

```
NEXT
```

STEP that does not descend. A CALL or RST runs to completion and stops at the return address instead of stepping into it; anything else is a plain single step. It is a temporary breakpoint at the return plus a RUN, so the callee is LIVE -- it can use the console, and ^E (ATTN) or ^C stops it.

```
N           over the CALL/RST at PC (else single-step)
```

RUN — `R[UN]`

```
RUN [addr]
```

Start the machine. `RUN <addr>` is EXAMINE + RUN -- it loads the PC first, exactly as you would on the panel.

If a unit holds the console, the GUEST GETS THE KEYBOARD -- every key, including ^C, which a CP/M program is entitled to read. The way back is ATTN (^E), which the host takes before the guest is ever offered the byte, so the guest cannot disable it. ATTN STOPS the machine -- nothing executes while this prompt is up -- but it does not DISTURB it: ATTN is not RESET and not POWER, so every register, every byte and every disk survives, and a bare RUN resumes at the exact instruction. Stopped is not lost, and the debugger is at its most useful here: REGS, EXAMINE, DUMP, DISASM and STEP all work at this prompt.

IT RUNS FLAT OUT unless the CPU card has a crystal. `clock_hz` defaults to 0, so a cassette that took a real Altair 110 seconds comes off in about one. `SET cpu0 clock_hz=2000000` buys back the 2 MHz machine AND its 110 seconds. What the guest sees is identical either way -- the tape still costs the same T-states -- so the crystal buys period FEEL, not period behaviour.

With no console connected there is no keyboard to hand over, and nothing to pace against: it simply runs, ^C stops it. Either way it stops on a breakpoint or on a HLT nothing can wake, and it ALWAYS says which.

```
RUN F800    boot the monitor PROM
RUN         carry on from wherever the PC is
```

HISTORY — `H[ISTORY]`

```
HISTORY [n]
```

The last n BUS CYCLES the machine ran, oldest first -- a flight recorder that is always running while the machine runs, so it already holds the run-up to a breakpoint or a crash when you ask. n is a count, so it is decimal; bare HISTORY shows the last 16. Each line is a cycle, not an instruction, and a DMA transfer's cycles are in there too.

```
HISTORY         the last 16 cycles
HISTORY 100     the last hundred
```

MOUNT — `M[OUNT]`

```
MOUNT <id>[:<u>] <file> [WP]
```

Put a disk in a drive, a tape in a recorder, or an image in a ROM socket. WP is the write-protect tab: the guest may read it and may not write it. RO is accepted and means the same -- it is the word for a ROM, which has no tab to move.

A NAME IS CASE-BLIND, and you may leave off what carries no information: the trailing index when only one such card is in the machine, and the unit when the card has only one you could mount into. Anything genuinely plural you must say, and it will tell you so.

```
MOUNT dsk0:drive0 disks/cpm.dsk
MOUNT dsk0:drive1 disks/master.dsk WP
MOUNT mem0:rom0 roms/monitor.bin
MOUNT ACR tape.bin      the one cassette, its one tape: acr0:tape
```

SHOW MOUNTS is the other half of this command: every socket in the machine, what is in it, and which are still empty. UNMOUNT takes it back out. A path is resolved as SHOW PATHS describes -- what you TYPE is relative to your shell.

BREAK — `B[REAK]`

```
BREAK [<addr> [IF <expr>] | MEM R|W <addr> | IO R|W <port>] [TRACE ON|OFF]
```

Bare BREAK lists them. Only the first kind is about the CPU at all -- the other two watch BUS CYCLES, so they catch a DMA transfer too, and they work unchanged on any processor.

```
BREAK FF13       stop when PC gets there
BREAK 2C00-2CFF  ...anywhere in a range
BREAK MEM W 100  stop when anything WRITES 0100
BREAK IO R 10    stop on an IN from port 10
```

A plain address breakpoint may carry a CONDITION -- IF over the registers -- and stops only when it holds. A bare word that names a register IS that register, so a literal is written with a leading zero (0A is ten, A is the accumulator). == != < > <= >= compare; && || combine; & | mask.

```
BREAK 100 IF A==0
BREAK 100 IF HL==8000 && Z==1
BREAK 100 IF (A&0F)==0
```

TRACE ON|OFF makes it a TRACEPOINT: instead of stopping, it turns TRACE on or off and the machine RUNS ON. Two of them trace a REGION and nothing else -- which is how you trace one subroutine out of a program that would otherwise bury you. Unlike IF, this works on the MEM and IO kinds too, because it reads no registers. TRACE ON at an address traces the instruction AT it; TRACE OFF does not -- the region is [on, off).

```
BREAK 2C00 TRACE ON        start tracing when PC gets to 2C00
BREAK 2C40 TRACE OFF       ...and stop again at 2C40
BREAK MEM W 2000 TRACE ON  start when anything writes 2000 (that write is
                           the first line -- a trace shows its own reason)
BREAK 200 IF HL==8000 TRACE ON    conditional, and still does not stop
```

Where the trace GOES is TRACE's business, not the tracepoint's: set it up with TRACE ON MASK=..., then TRACE OFF to arm it without emitting. An unconfigured tracepoint traces to the console.

CONFIG — `C[ONFIG]`

```
CONFIG LOAD <f.toml> | CONFIG SAVE <f.toml>
```

THE MACHINE, NOT WHAT IT IS DOING. SAVE writes the hardware you are actually running -- which cards, in what order, every property SET can write, what each unit is CONNECTed to, what is MOUNTed in each socket, and the startup list. It is the same format you would write by hand, and the same one a built-in is written in, so a saved machine is a first-class machine: LOAD it back, or name it on the command line, and you get exactly what you saved.

IT DOES NOT SAVE STATE, and that is not a gap to be filled: a machine file describes hardware, and none of this is hardware. NOT saved --

```
RAM             what you DEPOSITed is gone. LOAD/SAVE <file> <range> is for
                memory, and it is a separate file for a reason.
the registers   PC included, so a LOADed machine has not started.
breakpoints     nor tracepoints, nor where TRACE was pointed.
CONSOLE         attn and the transforms are the HOST's terminal, not a card
                in the backplane. They survive CONFIG LOAD untouched.
```

A SAVE IS A READ: it asks every property for its value and writes to nothing.

LOAD IS THE WHOLE MACHINE, so it REPLACES the one you have: the cards you had are out of the backplane, the new ones are in, and it is powered up and running its startup list -- a file whose startup says RUN comes up running. Naming that same file on the command line does the identical thing; there is one road.

AND IT IS ALL OR NOTHING. The machine is built off to one side first, so a file that will not load -- a key that does not parse, a disk image that is not there -- leaves you exactly where you were. What you do not get back is the machine you REPLACED: there is no undo but the file you saved it to.

```
CONFIG SAVE machines/mine.toml
CONFIG LOAD machines/mine.toml      ...and this is how you get it back
```

SET — `SE[T]`

```
SET <id> <k>=<v>
```

SHOW lists every property, its value, and whether it can be set while the machine runs. A property's base is its own: a port is hex, a baud rate is decimal.

```
SET mem0 fill=zero
SET mem0 phantom=read
```

SHOW — `SH[OW]`

```
SHOW <id>|BUS [MAP|IO|IRQ|CONTENTION]|ROMS|MOUNTS|PATHS|CONSOLE|SYMBOLS|MACHINE
```

```
SHOW mem0        regions and properties
SHOW BUS MAP     who decodes what, and what floats
SHOW BUS IRQ     VI0-VI7: who is strapped where, who is pulling, who wins
SHOW MOUNTS      every disk, tape and ROM in the machine, and what is in it
SHOW PATHS       what a path resolves against -- and there is more than one answer
SHOW CONSOLE     which unit holds the keyboard, and its transforms
SHOW SYMBOLS     the loaded symbols (SHOW SYMBOLS SIO* filters); load them with SYMBOLS
SHOW ROMS        the ROM images built into this binary, and where each came from
```

DEPOSIT — `DE[POSIT]`

```
DEPOSIT <addr> <bytes...>
```

The front-panel switch. Runs a REAL bus write, so if no board decodes the address the byte is simply gone -- and DEPOSIT says so rather than lying.

```
DE 100 C3 00 F8
```

EXAMINE — `EX[AMINE]`

```
EXAMINE [<addr>]
```

One byte: hex, ASCII, and the bits as the panel's LEDs showed them. Bare EXAMINE is the panel's EXAMINE NEXT -- it steps one byte. Its cursor is its own; a DUMP does not move it.

```
EX 100       0100  C3  .  11000011
EX           and the next byte, and the next
```

IN — `I[N]`

```
IN <port>
```

Runs a REAL IN cycle, with real side effects: an IN from a UART's data port consumes the byte and the guest never sees it. To look without touching, use WHO IO . Reports whether anybody actually answered.

```
I 10         port 10 -> FF   (nobody answered -- the bus floated it)
```

OUT — `O[UT]`

```
OUT <port> <byte>
```

Runs a REAL OUT cycle. Says so if no board decodes the port.

```
O 10 41
```

LOAD — `L[OAD]`

```
LOAD <file> [AT <addr>] [FORMAT=BIN|HEX] [ROM]
```

Put a file into memory. There are two kinds of file and they differ in ONE way -- whether the file knows where it goes.

```
HEX  Intel HEX. ASCII text, and it CARRIES ITS OWN ADDRESSES, so it needs
     no AT. Each line is one record: a ':', a length, the address, a type
     (00 data, 01 end-of-file), the bytes, and a checksum. Every checksum
     is verified and a bad one FAILS the load and names the record -- a
     half-loaded program is a miserable thing to debug.
       :10010000C300F8AF32004D3E0132014D76C9AA5509
       :00000001FF
        ^^^^^^^^^^                              ^^ checksum: the record
        |  |    |                                  sums to zero, byte-wise
        |  |    type 00 = data (01 = end of file)
        |  load address: these bytes go at 0100
        10 = sixteen data bytes follow
```

```
BIN  A flat binary: bytes, and nothing else. It carries NO addresses, so
     it cannot say where it goes and AT is REQUIRED. Without one, LOAD
     refuses rather than guess.
```

WHICH ONE IS DECIDED BY THE FILE'S CONTENTS, not its name -- Intel HEX announces itself, and a .bin full of HEX text is still HEX. FORMAT= overrides that when the sniff is wrong; it always wins.

AT MEANS 'PUT IT HERE', for both kinds. On a HEX file it relocates: the file's FIRST data record lands at AT and everything else moves by the same amount, wrapping at FFFF (a file whose first record is F000, loaded AT 0, puts its F800 record at 0800). Without AT, a HEX file loads where it says.

ROM is the PROM burner. LOAD writes through the bus, so a ROM never takes it -- a bus write cannot program a PROM on real hardware either, and LOAD says how many bytes landed nowhere rather than half-loading in silence. ROM goes behind the bus, straight into whichever chip answers reads at that address: the operator pulling the chip and putting it in a programmer. It is why the operator can write a ROM and the guest cannot.

```
LOAD dbl.hex                     where the file says, through the bus
LOAD monitor.bin AT F000 ROM     a flat binary, burned into the ROM there
LOAD prog.hex AT 100             relocate: first record goes to 0100
LOAD odd.txt AT 0 FORMAT=HEX     it IS hex, whatever it is called
```

SAVE — `SA[VE]`

```
SAVE <file> <range> [FORMAT=BIN|HEX]
```

Memory to a file, through the bus -- so what you get is what the CPU would read, ROM included. The range is what to save; a byte nobody drives reads FF.

THE NAME DECIDES THE FORMAT, and this is the other half of LOAD's rule rather than the same one: LOAD can open the file and see what it IS, and SAVE cannot -- the file does not exist yet. So a name ending .HEX writes Intel HEX and anything else writes a flat binary. FORMAT= says it outright when the name would guess wrong, and always wins.

```
SAVE out.hex 0-FFF               Intel HEX, by its name
SAVE out.bin F800-FFFF           a flat binary, by its name
SAVE out.dat 0-FFF FORMAT=HEX    hex, though it is not called .hex
```

FILL — `F[ILL]`

```
FILL <range> <byte>
```

```
FILL 0-3FF 00
```

SEARCH — `SEA[RCH]`

```
SEARCH <range> <bytes...>|"str"
```

```
SEA 0-FFFF C3
SEA 0-FFFF "CP/M"
```

COMPARE — `COM[PARE]`

```
COMPARE <range> <addr>|<file>
```

MOVE — `MOV[E]`

```
MOVE <range> <dest>
```

WHO — `W[HO]`

```
WHO <addr> | WHO IO <port>
```

Who WOULD answer -- it looks without running a cycle, so nothing is consumed and no card is poked. Reports contention, and reports PHANTOM*.

```
WHO FF00
WHO IO 10
```

BOARDS — `BO[ARDS]`

```
BOARDS [LIST]|TYPES|ADD <type> <id> [k=v...]|REMOVE <id>
```

The backplane: what is in it, what each card answers to, and what is in its sockets. A bare BOARDS lists them. RAM and ROM are named separately, and a ROM range says which image is in it -- an empty socket decodes nothing, so it is not in the memory column at all; it is in UNITS, marked (empty).

```
BOARDS                  the backplane
BOARD                   the same thing: a prefix of BOARDS
BOARDS TYPES            every card, and its properties
BOARDS ADD memory mem0
```

REGS — `RE[GS]`

```
REGS | SET REG <r>=<v>
```

The flags are registers too, so SET REG CY=1 works. A register value is on the wire, so it is HEX.

```
REGS
SET REG A=3F
SET REG PC=FF00
```

REGION — `REGI[ON]`

```
REGION ADD <id> type=ram|rom at=<addr> [size=|mount=]
```

A region is a POPULATED part of a card. What is not covered by one is an empty socket: it decodes nothing and floats to FF. `at` is an address, so it is hex; `size` is a size, so it is decimal, and K/M work.

```
REGI ADD mem0 type=ram at=0 size=48K
REGI ADD mem0 type=rom at=FF00 mount=builtin:dbl
```

DISASM — `DI[SASM]`

```
DISASM [<addr>|<range>] [n] [CPU=8080]
```

It needs an INSTRUCTION SET, not a CPU -- so it works on an empty backplane. You normally never type CPU=: the active core says what it speaks, and DISASM asks it. It PEEKS, so it cannot consume a byte from a UART in the range.

```
DI FF00      sixteen instructions of the boot PROM
DI           carry on from there
DI 0-2F      exactly that range
DI FF00 CPU=8080   when there is no CPU in the machine to ask
```

SYMBOLS — `SY[MBOLS]`

```
SYMBOLS LOAD <file> [REPLACE] | SYMBOLS CLEAR
```

Load an assembler's symbols so you can BREAK, DUMP and EXAMINE by NAME instead of by hex, and SHOW SYMBOLS to read the table. Two file kinds, and one absolute rule:

```
.PRN / .LST   an assembler LISTING -- CP/M ASM, Microsoft M80, DR MAC. It marks
              an EQU, so a constant is told apart from a program label.
.SYM          the CP/M symbol file DR MAC/RMAC write and SID reads. A flat list
              of name=value, no label/constant distinction. (L80 writes no .SYM.)
```

ADDRESSES MUST BE ABSOLUTE. A relocatable M80 listing is refused, by the line -- link it and load the .SYM, or assemble to an absolute origin.

LOAD merges (the newest of a clashing name wins, and it says how many); REPLACE clears first; CLEAR empties the table. A file named in a machine's startup is reloaded on CONFIG LOAD and round-trips through CONFIG SAVE.

```
SYMBOLS LOAD prog.SYM
SYMBOLS LOAD roms/ALTMON/ALTMON.PRN
SYMBOLS CLEAR
```

UNMOUNT — `U[NMOUNT]`

```
UNMOUNT <id>:<u>
```

The socket is then EMPTY -- those pages float to FF, exactly as a card with no chip in it does.

```
U dsk0:drive0
```

DISCONNECT — `DISC[ONNECT]`

```
DISCONNECT <id>:<u>
```

The line then goes nowhere. NOT an error: an unconnected 6850 sits there with TDRE set forever, and a program that writes to it works fine and talks to nobody -- which is exactly what the card does with no cable in it.

```
DISC sio0:b
```

CONSOLE — `CONS[OLE]`

```
CONSOLE [<k>=<v>...]
```

The host's terminal: what it is, who holds it, and how you get back from it. Bare CONSOLE shows it; CONSOLE k=v sets it. (SHOW CONSOLE and SET CONSOLE are the same thing said the long way.)

ATTN is the key that takes the keyboard BACK from a running guest. The host intercepts it before the guest is ever offered the byte, so the guest cannot disable it -- and that is why it must not be a key the guest needs.

```
CONSOLE           what it is set to, and which unit holds it
CONSOLE attn=1D   make it ^]  (hex: it is a byte on the wire)
```

To choose WHICH unit the console is wired to, that is CONNECT.

CONNECT — `CONN[ECT]`

```
CONNECT <id>:<u> <endpoint>
```

PLUG IN THE OTHER END OF THE CABLE. A unit is a socket on the back of a card -- one of the 2SIO's two ports, say; an ENDPOINT is the thing at the far end of the cable, on the HOST side of the machine. It is not a card, it has no address, and the guest cannot see it: the 6850 clocks bytes the same way whether the wire ends at your terminal, a telnet session, a real RS-232 port, or nothing at all. No card in the machine knows what any of these words mean.

Endpoints: {endpoints}

```
console     the host's terminal -- the keyboard and screen you are typing at
null        a cable to nowhere: writes vanish, reads never yield a byte
loopback    the unit's own transmit wired back to its receive, for testing
scripted    a terminal with a caller in place of a human -- what the MCP tools
            and the test suite type into. No tty need exist.
socket:     PORT alone LISTENS: that is the telnet-in case. HOST:PORT CALLS OUT.
serial:     a real port on this host. It is opened at 9600 8N1 and then
            immediately re-programmed by the card, which is the only thing that
            knows what it is strapped to.
```

Exactly ONE unit may hold the console; connecting a second STEALS it and says who from. Two boards reading one keyboard would each get half the characters.

```
CONN sio0:a console
CONN sio0:b null
CONN sio0:b loopback
CONN sio0:b socket:2323            `telnet localhost 2323` now reaches the guest
CONN sio0:b socket:bbs.example:23  the guest dials OUT, to somebody else's port
CONN sio0:b serial:/dev/tty.usbserial-AL009KFH    a real cable, real hardware
CONN sio0:b serial:COM3                           ...the same, on Windows
```

DISCONNECT takes the cable out again; SHOW CONSOLE says which unit holds it.

RESET — `RES[ET]`

```
RESET [CPU]
```

A reset does NOT clear memory. Only removing power does that -- see POWER. RESET CPU is a debugging convenience, NOT a real signal: no wire on the backplane resets the processor and nothing else.

POWER — `P[OWER]`

```
POWER
```

Power cycle. THE ONLY THING THAT LOSES RAM -- a RESET does not, because on real hardware it does not.

TRACE — `T[RACE]`

```
TRACE ON|OFF [file] [MASK=IN,OUT,IRQ,DMA,CONTENTION]
```

Log every BUS CYCLE while the machine runs -- to the console, or to a file. A cycle, not an instruction: MR/MW are memory, IN/OUT are I/O, INTA is an interrupt acknowledge, and a granted DMA master's cycles are tagged [DMA]. This watches the same stream every board sees, so it is not a CPU feature and works unchanged on any processor.

MASK keeps only the cycles you name (no MASK keeps all): IN, OUT, IRQ, DMA, CONTENTION. A cycle is kept if it is any of them -- MASK=DMA is every cycle a master drove, whatever its type.

```
TRACE ON                   every cycle, to the console
TRACE ON run.log           ...to a file
TRACE ON MASK=IN,OUT       just the port traffic
TRACE OFF
```

TRACE OFF stops the tracing but REMEMBERS where it was going -- a later TRACE ON, or a tracepoint, resumes to the same file and mask. That is what lets you aim a tracepoint at a file: TRACE ON run.log MASK=DMA, then TRACE OFF to arm it without emitting, then BREAK TRACE ON. See BREAK.

NOBREAK — `NO[BREAK]`

```
NOBREAK [id]
```

Bare NOBREAK clears them all. An id is not on the wire, so it is decimal.

```
NOBREAK 2
NOBREAK
```

HELP — `HE[LP]`

```
HELP [<command>]
```

Bare HELP lists the commands and nothing else -- the whole set on a few lines, which is what you want when you are hunting for the name. HELP with a command gives the usage and the examples.

```
HELP         the list
HELP DUMP    the detail
?            the same as HELP
```

QUIT — `Q[UIT]`

```
QUIT
```

Boards and their parameters

Every key below is a key you may write in a machine file, and the *same* key you may `SET` at the monitor prompt. That is not a coincidence and it is not a convention: a board's properties **are** its TOML schema, so there is nothing here that could disagree with the program.

Numbers follow the one rule: **on the wire → hex, never on the wire → decimal.** A port is hex; a baud rate and a drive count are decimal. The defaults below are printed in each property's own base.

Type	What it is
`memory`	RAM/ROM card: a list of regions, PHANTOM*, and five banking schemes
`8080`	MITS 88-CPU: an 8080A at 2 MHz. Decodes nothing -- it drives the bus
`z80`	Generic Z80 CPU card. Decodes nothing -- it drives the bus. The 88-CPU's twin, with a Z80 core
`2sio`	MITS 88-2SIO: two 6850 ACIAs, units 'a' and 'b'. Four ports at BASE+0..3
`sio`	MITS 88-SIO: one COM2502 UART, unit 'tty'. Two ports at BASE+0..1. INVERTED status bits
`dcdd`	MITS 88-DCDD: 8" hard-sector floppy, up to 16 drives. Three ports at BASE+0..2. INVERTED status bits
`mds`	MITS 88-MDS: 5.25" minidisk, 4 drives. Same three ports as the dcdd -- but 300 RPM, 64 us/byte, and a motor that stops after 6.4 s
`acr`	MITS 88-ACR: cassette. An 88-SIO B + an FSK modem, unit 'tape'. Brings the REWIND verb
`fp`	Altair front panel: the SENSE switches at port FF (read-only), and the lamps
`virtc`	MITS 88-VI/RTC: vectored interrupts (VI0-VI7 -> RST n) and a real-time clock. One port at FE
`hostbridge`	Host Bridge: guest <-> host file transfer, sandboxed. OUR OWN CARD, not a period one. Two ports at BASE+0..1. R.COM/W.COM/HDIR.COM

`memory`

RAM/ROM card: a list of regions, PHANTOM*, and five banking schemes

`[[board.region]]` — a list you may add

Key	Kind	Legal	Meaning
`type`	enum	`ram` | `rom`	RAM, or ROM (which needs a `mount`, unless you want an empty socket)
`at`	int	`0x0` .. `0xFFFF`	Where it starts. An address: 0000, F800
`size`	int	`1` .. `65536`	How much. Decimal, and it takes a suffix: 48K, 1024, 2M
`mount`	string	text	The ROM image. A file (relative to THIS FILE), or builtin:

Board properties

Key	Kind	Default	Legal	Meaning
`honors_phantom`	enum	`all`	`none` | `read` | `all`	A JUMPER. Another board pulls PHANTOM* -- do I switch off? none | read | all
`phantom`	enum	`all`	`none` | `read` | `all`	What I ASSERT over my rom regions: none | read | all
`bank_type`	enum	`none`	`none` | `eram` | `vram` | `cram` | `hram` | `b810`	none|eram|vram|cram|hram|b810 -- five real cards, no two alike
`banks`	int	—	—	how many banks this card has. The card decides: it follows bank_type **(read-only — not a key you may set)**
`bank`	int	`0`	`0` .. `15`	The live bank
`fill`	enum	`random`	`zero` | `random`	RAM contents at power-on: zero | random (real RAM is not zeroed)
`seed`	int	`1`	any	Seed for fill=random. Goes in the snapshot, or replay is dead.
`pages`	string	—	—	the composite page map -- which pages this card answers for. Derived from the regions you declared **(read-only — not a key you may set)**

`8080`

MITS 88-CPU: an 8080A at 2 MHz. Decodes nothing -- it drives the bus

Units: `8080` (cpu)

Board properties

Key	Kind	Default	Legal	Meaning
`clock_hz`	int	`0`	`0` .. `100000000`	Crystal on the card. 0 runs flat out -- as fast as the host can.
`idle`	bool	`true`	`on` | `off`	Stand down when the guest is only polling an empty keyboard. On by default -- the guest cannot tell, and a prompt stops burning a core.
`achieved_hz`	int	—	—	LIVE: T-states per real second the run loop last reached -- the crystal you got, beside the one you asked for. Read-only; 0 until it has run. **(read-only — not a key you may set)**

`z80`

Generic Z80 CPU card. Decodes nothing -- it drives the bus. The 88-CPU's twin, with a Z80 core

Units: `z80` (cpu)

Board properties

Key	Kind	Default	Legal	Meaning
`clock_hz`	int	`0`	`0` .. `100000000`	Crystal on the card. 0 runs flat out -- as fast as the host can.
`idle`	bool	`true`	`on` | `off`	Stand down when the guest is only polling an empty keyboard. On by default -- the guest cannot tell, and a prompt stops burning a core.
`achieved_hz`	int	—	—	LIVE: T-states per real second the run loop last reached -- the crystal you got, beside the one you asked for. Read-only; 0 until it has run. **(read-only — not a key you may set)**

`2sio`

MITS 88-2SIO: two 6850 ACIAs, units 'a' and 'b'. Four ports at BASE+0..3

Units: `a` (serial), `b` (serial)

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0x10`	`0x0` .. `0xFC`	Base address. The card decodes four ports: BASE+0 .. BASE+3

Unit `a` — `[board.unit.a]`

Key	Kind	Default	Legal	Meaning
`baud`	int	`9600`	`50` .. `76800`	Line rate. A JUMPER on the real card -- software cannot change it, and there is no free-running setting: the rate paces the line
`interrupt`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where this channel's IRQ is jumpered: none | int | vi0..vi7 *(interrupt strap)*
`dcd`	enum	`ground`	`ground` | `wired`	/DCD pin: grounded on the card, or wired to the connector
`cts`	enum	`ground`	`ground` | `wired`	/CTS pin: grounded on the card, or wired -- and then it gates the transmitter
`lines`	string	—	—	Live pin state (read-only). CAPITALS = asserted. in: DCD CTS, out: RTS BRK **(read-only — not a key you may set)**
`connect`	string	`null`	text	The endpoint on the other end of the line (CONNECT sets this)

Unit `b` — `[board.unit.b]`

Key	Kind	Default	Legal	Meaning
`baud`	int	`9600`	`50` .. `76800`	Line rate. A JUMPER on the real card -- software cannot change it, and there is no free-running setting: the rate paces the line
`interrupt`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where this channel's IRQ is jumpered: none | int | vi0..vi7 *(interrupt strap)*
`dcd`	enum	`ground`	`ground` | `wired`	/DCD pin: grounded on the card, or wired to the connector
`cts`	enum	`ground`	`ground` | `wired`	/CTS pin: grounded on the card, or wired -- and then it gates the transmitter
`lines`	string	—	—	Live pin state (read-only). CAPITALS = asserted. in: DCD CTS, out: RTS BRK **(read-only — not a key you may set)**
`connect`	string	`null`	text	The endpoint on the other end of the line (CONNECT sets this)

`sio`

MITS 88-SIO: one COM2502 UART, unit 'tty'. Two ports at BASE+0..1. INVERTED status bits

Units: `tty` (serial)

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0x0`	`0x0` .. `0xFE`	Base address -- MUST BE EVEN. Control at BASE, data at BASE+1
`rev`	enum	`1`	`0` | `1`	Board revision. 1 = the errata mod done at the factory (see the .md)
`baud`	int	`9600`	`50` .. `25000`	Line rate. A JUMPER on the real card -- software cannot change it
`data_bits`	int	`8`	`5` .. `8`	Data bits per character. The NDB1/NDB2 pads
`stop_bits`	int	`1`	`1` .. `2`	Stop bits. The NSB pad: GND = 1, +V = 2
`parity`	enum	`none`	`none` | `odd` | `even`	The NPB/POE pads: none | odd | even
`in_int`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the IN pad is soldered (RX): none | int | vi0..vi7 *(interrupt strap)*
`out_int`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the OUT pad is soldered (TX): none | int | vi0..vi7 *(interrupt strap)*
`connect`	string	`null`	text	The endpoint on the other end of the line (CONNECT sets this)

`dcdd`

MITS 88-DCDD: 8" hard-sector floppy, up to 16 drives. Three ports at BASE+0..2. INVERTED status bits

Units: `drive0` (disk), `drive1` (disk), `drive2` (disk), `drive3` (disk)

`[[board.drive]]` — a list you may add

Key	Kind	Legal	Meaning
`unit`	int	`0` .. `3`	Which drive on the daisy chain
`mount`	string	text	The disk image to put in it. Relative to THIS FILE.
`readonly`	bool	`on` | `off`	The write-protect tab. The DRIVE senses it, so the guest is never told: it writes, the head is inhibited, the bytes go nowhere
`media`	enum	`8in` | `fdc8mb`	Force the format instead of probing the image's size

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0x8`	`0x0` .. `0xFD`	Base address. The card decodes three ports: BASE+0 .. BASE+2
`drives`	int	`4`	`1` .. `16`	Drives on the daisy chain
`interrupt`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the card's interrupt is soldered *(interrupt strap)*

`mds`

MITS 88-MDS: 5.25" minidisk, 4 drives. Same three ports as the dcdd -- but 300 RPM, 64 us/byte, and a motor that stops after 6.4 s

Units: `drive0` (disk), `drive1` (disk), `drive2` (disk), `drive3` (disk)

`[[board.drive]]` — a list you may add

Key	Kind	Legal	Meaning
`unit`	int	`0` .. `3`	Which drive on the daisy chain
`mount`	string	text	The disk image to put in it. Relative to THIS FILE.
`readonly`	bool	`on` | `off`	The write-protect tab. The DRIVE senses it, so the guest is never told: it writes, the head is inhibited, the bytes go nowhere
`media`	enum	`minidisk`	Force the format instead of probing the image's size

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0x8`	`0x0` .. `0xFD`	Base address. The card decodes three ports: BASE+0 .. BASE+2
`drives`	int	`4`	`1` .. `4`	Drives on the daisy chain
`interrupt`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the card's interrupt is soldered *(interrupt strap)*
`motor`	enum	`free`	`free` | `real`	free: always at speed (default). real: 1 s spin-up, and it stops after 6.4 s

`acr`

MITS 88-ACR: cassette. An 88-SIO B + an FSK modem, unit 'tape'. Brings the REWIND verb

Units: `tape` (tape)

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0x6`	`0x0` .. `0xFE`	Base address -- MUST BE EVEN. Control at BASE, data at BASE+1
`rev`	enum	`1`	`0` | `1`	Board revision. 1 = the errata mod done at the factory (see the .md)
`baud`	int	`300`	`50` .. `25000`	Line rate. A JUMPER on the real card -- software cannot change it
`data_bits`	int	`8`	`5` .. `8`	Data bits per character. The NDB1/NDB2 pads
`stop_bits`	int	`1`	`1` .. `2`	Stop bits. The NSB pad: GND = 1, +V = 2
`parity`	enum	`none`	`none` | `odd` | `even`	The NPB/POE pads: none | odd | even
`in_int`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the IN pad is soldered (RX): none | int | vi0..vi7 *(interrupt strap)*
`out_int`	enum	`none`	`none` | `int` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the OUT pad is soldered (TX): none | int | vi0..vi7 *(interrupt strap)*

Unit `tape` — `[board.unit.tape]`

Key	Kind	Default	Legal	Meaning
`mode`	enum	`play`	`play` | `record`	The button that is down on the recorder: play | record

`fp`

Altair front panel: the SENSE switches at port FF (read-only), and the lamps

Board properties

Key	Kind	Default	Legal	Meaning
`sense`	int	`0x0`	`0x0` .. `0xFF`	The SENSE switches, SA8..SA15 -- what IN 0FFH reads
`data`	int	`0x0`	`0x0` .. `0xFF`	The DATA switches, SA0..SA7. Not readable by the guest -- see the .md

`virtc`

MITS 88-VI/RTC: vectored interrupts (VI0-VI7 -> RST n) and a real-time clock. One port at FE

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0xFE`	`0x0` .. `0xFF`	Control port. 0xFE (376 octal) on the real card -- write only
`rtc_source`	enum	`line`	`line` | `clock`	RTC clock source jumper: the 60 Hz line, or 10 kHz off the 2 MHz clock
`rtc_divide`	enum	`1`	`1` | `10` | `100` | `1000`	RTC divider jumper: source frequency / 1, 10, 100 or 1000
`rtc_interrupt`	enum	`none`	`none` | `vi0` | `vi1` | `vi2` | `vi3` | `vi4` | `vi5` | `vi6` | `vi7`	Where the RTC's interrupt ("RI") is jumpered: none | vi0..vi7. Leave it `none` to run the PS2 package *(interrupt strap)*
`vi_enabled`	bool	—	—	LIVE: is the 88-VI structure enabled? (control bit 7; POC clears it) **(read-only — not a key you may set)**
`level_live`	bool	—	—	LIVE: is the current-level comparison in circuit? (control bit 3) **(read-only — not a key you may set)**
`rtc_pending`	bool	—	—	LIVE: has the RTC's interrupt flip-flop set? (cleared by control bit 4) **(read-only — not a key you may set)**
`current_level`	int	—	—	LIVE: the current interrupt level (control bits 0-2, ones-complement on the wire). Nothing at this level or below may interrupt while level_live **(read-only — not a key you may set)**

`hostbridge`

Host Bridge: guest <-> host file transfer, sandboxed. OUR OWN CARD, not a period one. Two ports at BASE+0..1. R.COM/W.COM/HDIR.COM

Board properties

Key	Kind	Default	Legal	Meaning
`port`	int	`0xB0`	`0x0` .. `0xFE`	Base port. Two ports: BASE+0 command/status, BASE+1 data
`hostdir`	string		text	The sandbox root. Guest names resolve here and CANNOT escape it. Empty = the shell's working directory
`hostdir_root`	string	—	—	LIVE: the sandbox root as RESOLVED -- the actual directory the guest is fenced into. Read-only; `hostdir` is what was written. **(read-only — not a key you may set)**
`readonly`	bool	`false`	`on` | `off`	Refuse OPEN_WRITE and DELETE -- the guest may read the host, not change it

The built-in machines

A built-in is a machine file that lives **inside the binary** — the same format you would write yourself, with nothing special about it. Name one and you get it in any directory on earth:

```
$ altairsim basic4k
```

`altairsim --list` prints this table, and `altairsim -x 'SHOW MACHINE' <name>` shows what is actually in one.

Machine	What it is
`4k`	The Altair as it actually left Albuquerque.
`altmon`	An Altair with a monitor in ROM and a terminal on it.
`basic4k`	The machine Altair 4K BASIC was sold to run on: a cassette in the ACR, a Teletype
`basic8k`	The machine Altair 8K BASIC was sold to run on: a cassette in the ACR, and a terminal
`default`	The machine you get when you name none: 56K, and the DBL boot PROM at FF00.
`minidisk`	The Altair Minidisk: an 88-MDS at 08, the MDBL boot PROM, and CP/M 2.2b on a 5.25" disk.
`ps2`	The machine MITS Programming System II ran on. It is `basic8k`'s CARDS -- same 2SIO, same
`ps2int`	MITS Programming System II, WITH INTERRUPTS. `ps2` with A9 down and an 88-VI/RTC in it.
`z80`	A minimal Z80 machine: a `z80` CPU, 64K of RAM, and a 2SIO console.